HYGO SMART HYGIENE MANAGEMENT SYSTEM

A MINI PROJECT REPORT

by

AJOE JOSE (VJC23CS014)
AJU ELDO (VJC23CS016)
JINCE JOSE (VJC23CS109)
KEVIN JACOB (VJC23CS123)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

MAY 2026

HYGO SMART HYGIENE MANAGEMENT SYSTEM

A MINI PROJECT REPORT

by

AJOE JOSE (VJC23CS014)
AJU ELDO (VJC23CS016)
JINCE JOSE (VJC23CS109)
KEVIN JACOB (VJC23CS123)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

under the guidance

of

Mrs. SANTHI M

Assistant Professor,CSE Department

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

MAY 2026

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

Department of Computer Science and Engineering

Vision

Moulding socially responsible and professionally competent Computer Engineers to adapt to the dynamic technological landscape

Mission

1. Foster the principles and practices of computer science to empower life-long learning and build careers in software and hardware development.

2. Impart value education to elevate students to be successful, ethical and effective problem-solvers to serve the needs of the industry, government, society and the scientific community.

3. Promote industry interaction to pursue new technologies in Computer Science and provide excellent infrastructure to engage faculty and students in scholarly research activities.

Program Educational Objectives

Our Graduates

1. Shall have creative aid critical reasoning skills to solve technical problems ethically and responsibly to serve the society.

2. Shall have competency to collaborate as a team member and team leader to address social, technical and engineering challenges.

3. Shall have ability to contribute to the development of the next generation of information technology either through innovative research or through practice in a corporate setting

4. Shall have potential to build start-up companies with the foundations, knowledge and experience they acquired from undergraduate education

Program Outcomes

1. **Engineering knowledge**:Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**:Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences

3. **Design / development of solutions**:Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural, societal and environmental considerations.

4. **Conduct investigations of complex problems**:Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**:Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society:**Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:**Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of and need for sustainable development.

8. **Ethics:**Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice

9. **Individual and team work:**Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings

10. **Communication:**Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and unread in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**:Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

1. Ability to integrate theory and practice to construct software systems of varying complexity
2. Able to Apply Computer Science skills, tools and mathematical techniques to analyse, design and model complex systems
3. Ability to design and manage small-scale projects to develop a career in a related industry.

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that the mini project work entitled **"HYGO SMART HYGIENE MANAGEMENT SYSTEM"** is a bonafide work done by **AJOE JOSE(VJC23CS014), AJU ELDO(VJC23CS016), JINCE JOSE(VJC23CS109)** and **KEVIN JACOB (VJC23CS123)** in partial fulfillment of the award of the Degree of **Bachelor of Technology in Computer Science & Engineering** from APJ Abdul Kalam Technological University, Thiruvananthapuram, Kerala during the academic year 2025-2026

Internal Supervisor **External Supervisor**

Mini Project Coordinator **Head of Department**

ACKNOWLEDGEMENT

First and foremost, We thank **God Almighty** for His divine grace and blessings in making all these possible. May He continue to lead us in the years. It is our privilege to render our heartfelt thanks to our most beloved Manager **Msg. Dr. PIOUS MALEKANDATHIL**, our Director **Rev. Fr. PAUL PARATHAZHAM** and our Principal **Dr. K K RAJAN** for providing us the opportunity to do this mini project during the sixth semester of our B.Tech degree course. We are deeply thankful to our Head of the Department, **Dr. AMEL AUSTINE** for his support and encouragement. We would like to express our sincere gratitude and heartfelt thanks to our Mini Project Guide **Mrs. SANTHI M**, Asst. Professor, Department of Computer Science and Engineering for her motivation , assistance and help for the project. We also express our sincere thanks to the Mini Project Coordinators **Dr. NEENU DANIEL** and **Ms. SWATHI VENUGOPAL**for their guidance and support.We also thank all the staff members of the Computer Science Department for providing their assistance and support. Last, but not the least, we thank all our friends and family for their valuable feedback from time to time as well as their help and encouragement.

DECLARATION

We undersigned hereby declare that the mini project report **"HYGO SMART HYGIENE MANAGEMENT SYSTEM"**, submitted for partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of the APJ Abdul Kalam Technological University is a bonafide work done under the supervision of **Mrs. SANTHI M**. This submission represents ideas in our own words and where ideas or words of others have been included, We have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or formed the basis for the award of any degree, diploma or similar title of any other University.

AJOE JOSE
AJU ELDO
Place : Vazhakulam **JINCE JOSE**
KEVIN JACOB

ABSTRACT

This project presents an intelligent IoT-based solution for public hygiene management, namely the Smart Public Toilet Management System (HYGO), designed to improve sanitation efficiency through real-time monitoring and data-driven decision-making. The system replaces traditional manual inspection and fixed cleaning schedules with a dynamic, demand-based cleaning model triggered by actual usage and odour levels. It utilizes MQ-135 gas sensors for odour detection and ultrasonic sensors for user count monitoring to continuously assess toilet conditions. When predefined thresholds are exceeded, the system sends instant alerts to municipal authorities, enabling timely assignment of cleaning staff through a web-based dashboard. Additionally, the system incorporates an AI/ML-based prediction model to optimize staff scheduling by analyzing previous days' usage patterns and predicting cleaning requirements in advance, allowing efficient and proactive staff allocation. The cleaning process is automatically verified using sensor data, ensuring task completion and enhancing accountability. The system also logs staff performance, providing transparency and improving operational efficiency. Developed using Python with Flask for backend and MySQL for database management, the system ensures reliable data handling and real-time updates. Through effective implementation, the system enhances public hygiene, optimizes resource utilization, ensures timely maintenance, and improves overall user satisfaction, demonstrating the impact of IoT and AI-driven automation in modern sanitation management.

Key Words :– IoT, Smart Toilet Management, Machine Learning, Usage Pattern Analysis, MQ-135 Sensor, Ultrasonic Sensor, Flask, MySQL, HYGO

Contents

List of Figures

List of Abbreviations

Abbreviation	Description
IoT	Internet of Things
ESP32	Espressif Microcontroller
MQ-135	Gas Sensor
Ultrasonic Sensor	Distance Sensor
AI	Artificial Intelligence
ML	Machine Learning
API	Application Programming Interface
UI	User Interface
UML	Unified Modeling Language
MVC	Model-View-Controller
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
Wi-Fi	Wireless Fidelity
DB	Database
MySQL	Relational Database Management System
Flask	Python Web Framework
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JavaScript	Scripting Language
REST API	Representational State Transfer
JSON	JavaScript Object Notation
Random Forest	Machine Learning Algorithm
VS Code	Visual Studio Code

Chapter 1

INTRODUCTION

In the modern world, maintaining public hygiene has become a critical concern, especially in high-traffic areas such as public toilets, where cleanliness directly impacts health, safety, and user satisfaction. Traditional sanitation systems largely depend on manual inspection and fixed clean- ing schedules, which often fail to reflect the actual usage and hygiene conditions of the facilities. This results in delayed maintenance, inefficient resource utilization, and poor hygiene standards [1, 2, 5]. Despite the growing need for smarter solutions, there is still a lack of intelligent systems that can monitor hygiene conditions in real time and enable data-driven decision-making for sanitation management.

This project proposes a solution in the form of the HYGO – Smart Hygiene Management System, an intelligent IoT-based platform designed to enhance public toilet maintenance through real-time monitoring and automation. The system utilizes sensors such as MQ-135 gas sensors for odour detection [6] and ultrasonic sensors for usage tracking to continuously assess hygiene condi- tions. It integrates a backend system developed using Python with Flask [8] and a MySQL database [7] to process sensor data, generate automatic cleaning alerts, and manage staff assignments through a web-based dashboard. Additionally, the system incorporates AI/ML-based predictive analytics using Scikit-learn [11] and Joblib [12], which analyze previous days' usage patterns to forecast cleaning requirements and optimize staff scheduling, ensuring proactive and efficient maintenance.

By leveraging IoT technology, real-time data processing, and intelligent prediction models [3, 4, 9], HYGO aims to transform traditional sanitation practices into a smart, demand-based system. The system also follows REST API principles [10] for efficient communication and adheres to software requirement standards [9] to ensure reliability. The system includes features such as automated cleaning verification, staff performance tracking, and comprehensive data logging for transparency and accountability. Designed with a user-friendly interface using React [13] and powered by ESP32 hardware [14], HYGO ensures efficient hygiene management, optimal resource utilization, and improved public health standards, making it a reliable and innovative

solution for modern sanitation challenges.

1.1 Problem Definition

Maintaining proper hygiene in public toilets is a significant challenge in modern urban environments, where high usage and inadequate maintenance can lead to poor sanitation conditions. Cleanliness in such facilities directly affects public health, safety, and user satisfaction [1, 2]. How- ever, existing sanitation systems primarily rely on manual inspection and fixed cleaning schedules, which do not accurately reflect real-time usage or hygiene conditions [5]. This often results in either over-cleaning or delayed cleaning, leading to inefficient resource utilization and unsatisfactory user experiences.

While some monitoring solutions exist, many lack real-time data collection, automated alert mechanisms, and intelligent decision-making capabilities [3, 4]. Additionally, there is minimal tracking of cleaning staff performance, resulting in a lack of accountability and transparency in sanitation management. The absence of data-driven insights makes it difficult for authorities to optimize cleaning schedules, predict usage patterns, or respond effectively to changing hygiene conditions. Consequently, public toilets often remain unhygienic, negatively impacting public perception and health standards [9].

There is a pressing need for an intelligent, real-time, and automated system that can continuously monitor hygiene conditions and enable efficient sanitation management. HYGO addresses this gap by integrating IoT-based sensors for odour and usage monitoring [6], real-time data process- ing using Flask and MySQL [8, 7], and AI/ML-based predictive analytics with Scikit-learn [11]. The system not only generates automatic cleaning alerts but also optimizes staff scheduling based on previous usage patterns, ensuring proactive maintenance. By incorporating performance tracking, automated verification, and a centralized dashboard using React [13], HYGO provides a comprehensive and scalable solution for improving public hygiene and operational efficiency.

1.2 Objective

The primary objective of the HYGO – Smart Hygiene Management System is to improve public sanitation by enabling real-time monitoring, intelligent decision-making, and automated cleaning management through an IoT-based platform [1, 2]. The key goals include:

Real-Time Hygiene Monitoring: The system utilizes IoT sensors such as MQ-135 gas sensors and ultrasonic sensors to continuously monitor odour levels and toilet usage, providing accurate and real-time hygiene status [6].

Automatic Cleaning Alerts: Generates instant alerts to authorities when hygiene conditions exceed predefined thresholds, ensuring timely cleaning and maintenance [5].

AI/ML-Based Predictive Scheduling: Incorporates machine learning techniques to analyze previous days' usage patterns and predict cleaning requirements, enabling efficient and proactive staff allocation [11, 12].

Staff Monitoring and Performance Tracking: Tracks cleaning activities, records response times, and calculates trust scores to ensure accountability and improve staff efficiency [9].

Centralized Dashboard System: Provides a web-based interface for authorities to monitor real-time data, assign staff, and view reports and analytics effectively using modern web technologies [13].

Database Integration: Ensures structured storage of sensor data, cleaning logs, staff records, and prediction results for efficient data management and analysis [7].

User-Friendly Interface: Designed to be simple, responsive, and easy to use, allowing authorities to manage sanitation operations efficiently.

By incorporating these features, HYGO enhances public hygiene, ensures timely maintenance, and optimizes resource utilization. Furthermore, the system promotes transparency and accountability while leveraging IoT and AI technologies to create a smart, data-driven sanitation management solution [3, 4].

1.3 Scope

The scope of the HYGO – Smart Hygiene Management System extends beyond its initial implementation, offering significant potential for future enhancements and integration with advanced technologies [1, 3]. The system can be effectively used by municipal authorities for public infrastructure management, enabling real-time monitoring and maintenance of hygiene conditions in public toilets, thereby improving sanitation standards [2, 5]. It can also be integrated into smart city frameworks to support centralized monitoring of multiple facilities and enable data-driven urban management [3, 4].

HYGO enhances workforce optimization by utilizing AI/ML-based predictive scheduling to analyze previous usage patterns and allocate cleaning staff efficiently, reducing workload imbalance and improving operational efficiency [11, 12]. Through continuous monitoring using IoT sensors such as MQ-135, the system enables real-time data collection and automated alert generation, minimizing the need for manual inspection and ensuring faster responses to hygiene issues [6]. Additionally, the system supports data-driven decision-making by storing and analyzing historical data, allowing authorities to optimize cleaning schedules and improve long-term sanitation planning [7].

The system is scalable and can be extended to various public facilities such as hospitals, railway stations, shopping malls, and educational institutions, making it a versatile hygiene management solution [9]. Furthermore, HYGO can be enhanced by integrating advanced AI models, mobile applications for staff, real-time notifications, and cloud-based analytics for large-scale deployment [11]. It also has the potential to incorporate features such as staff tracking, automated user feedback systems, integration with government sanitation platforms, and multilingual support for improved accessibility.

Overall, HYGO provides a scalable, intelligent, and efficient solution for modern sanitation challenges by leveraging IoT, real-time data processing, and predictive analytics, ultimately improving public hygiene standards, optimizing resource utilization, and promoting transparency and accountability in sanitation management [4, 10].

Chapter 2

Existing System

Despite the advancements in IoT-based sanitation technologies, existing solutions for public toilet hygiene management mainly focus on monitoring environmental parameters or providing basic alert systems rather than offering a fully automated and intelligent solution [1, 5]. While these systems contribute to improving hygiene conditions, they still have several limitations in terms of real-time decision-making, predictive analysis, and staff management. The following are some of the major existing systems:

2.1 IoT-Based Hygiene Monitoring System in Restroom

This system uses IoT sensors such as MQ-137 ammonia gas sensors and environmental sensors to monitor indoor air quality (IAQ), humidity, and temperature in public restrooms. The data is transmitted using MQTT protocol and stored in databases for real-time monitoring and analysis [6]. Alerts are generated when ammonia levels or IAQ exceed predefined thresholds, enabling timely cleaning actions. The system also analyzes correlations between environmental factors and hygiene conditions to improve sanitation management.

2.1.1 Advantages

- Provides real-time monitoring of ammonia levels and air quality.
- Enables remote monitoring using dashboards and notifications.
- Helps in data-driven decision making using historical analysis.
- Reduces unnecessary cleaning by triggering alerts only when needed.

2.1.2 Disadvantages

- Focuses mainly on monitoring and alerting, not full automation.

- Does not include AI/ML-based predictive scheduling.
- No proper staff performance tracking or accountability system.
- System accuracy depends heavily on sensor calibration and reliability.

2.2 LoRa-Based Smart Hygiene Monitoring System

This system uses LoRa (Long Range) communication technology to monitor hygiene conditions in public toilets over long distances with low power consumption. It integrates sensors such as ammonia gas sensors and occupancy sensors to detect odour levels and user presence [3]. The collected data is transmitted through LoRa modules to a central server or gateway, where it is processed and displayed on a monitoring dashboard. When hygiene parameters exceed predefined thresholds, alerts are generated and sent to authorities for cleaning actions.

2.2.1 Advantages

- Supports long-range communication without relying on Wi-Fi.
- Provides real-time alerts for hygiene issues.
- Reduces infrastructure dependency compared to Wi-Fi systems.
- Low power consumption, suitable for continuous monitoring.

2.2.2 Disadvantages

- Limited data transmission speed compared to Wi-Fi.
- Does not include advanced AI/ML-based prediction.
- No staff scheduling or performance tracking features.
- Limited real-time data processing capability.

2.3 General Smart toilets in Existing System

Several existing public toilet management systems rely on IoT-based monitoring and basic alert mechanisms to maintain hygiene [4, 9]. These systems typically use sensors to detect parameters such as odour levels, air quality, and user count, providing data to authorities for monitoring purposes. While these solutions offer some level of automation, they are primarily limited to observation and alert generation rather than complete intelligent management.

Some systems also include dashboards or mobile applications that allow authorities to view real-time data and receive notifications. However, these systems often depend on manual intervention for cleaning actions and lack advanced features such as predictive analysis and efficient staff management.

2.3.1 **Advantages**

- Enables real-time monitoring of hygiene conditions in public toilets.
- Enhances hygiene awareness through continuous sensor-based data collection.
- Supports remote monitoring via web dashboards and mobile applications.
- Generates automated alerts to ensure timely cleaning and maintenance.

2.3.2 **Disadvantages**

- Limited to basic monitoring and alert generation, lacking full automation.
- No AI/ML-based predictive analysis for optimizing cleaning schedules.
- Absence of intelligent staff allocation and performance tracking mechanisms.
- Dependence on manual decision-making after receiving alerts.
- Limited integration of multiple features such as monitoring, prediction, and verification in a single system.
- Many systems rely heavily on continuous internet connectivity, reducing reliability in certain environments.

These limitations highlight the need for a **comprehensive** and **intelligent** system like **HYGO**, which integrates real-time IoT monitoring, AI/ML-based predictive scheduling, automated alert generation, and staff performance tracking to provide an efficient and scalable solution for public hygiene management.

Chapter 3

REQUIREMENT ANALYSIS

3.1 Product Perspective

The product perspective of the HYGO – Smart Hygiene Management System focuses on its role as an intelligent, IoT-based solution for maintaining hygiene in public toilets through real-time monitoring and automated decision-making [1]. Unlike traditional systems that rely on fixed cleaning schedules, HYGO introduces a smart, demand-based approach by continuously analyzing hygiene conditions using sensor data. The system integrates IoT technology, a web-based dashboard, database management and AI/ML-based predictive analytics [11] to provide a comprehensive hygiene management solution. It acts as a centralized platform that enables authorities to monitor toilet conditions in real time, receive automatic cleaning alerts, manage staff efficiently and analyze reports for better decision-making. By utilizing advanced technologies such as IoT sensors and machine learning models like Random Forest [11], HYGO enhances efficiency, accountability and public hygiene standards.

3.2 Product Functions

HYGO is designed to provide a wide range of functionalities to support real-time hygiene monitoring, automated alerts and intelligent resource management.

3.2.1 User Application Functions:

Input Collected from Users:

- User Login/Authentication (Admin & Staff)
- Real-time odour level data from MQ-135 sensor
- Toilet usage data from ultrasonic sensor
- Cleaning activity updates from staff

- Staff details and attendance information
- Complaint submissions related to hygiene issues
- Historical data for analytics and prediction

Output Provided to Users:

- Real-time hygiene status (odour level and usage count)
- Automatic cleaning alerts when thresholds are exceeded
- Cleaning task assignments for staff
- Staff performance and trust score updates
- Dashboard visualization of reports and analytics
- Complaint status updates
- Predicted usage patterns and recommended cleaning time (AI/ML)

3.3 User Classes and Characteristics

Administrator (Admin)

- Monitor real-time hygiene conditions through the dashboard
- Receive automatic cleaning alerts when thresholds are exceeded
- Assign cleaning tasks to staff based on alerts
- Set and update hygiene threshold values
- View cleaning reports and analytics
- Monitor staff performance and trust scores
- Manage complaints and update their status

Cleaning Staff (Staff)

- Receive cleaning alerts and assigned tasks from the system
- Perform cleaning activities based on alerts
- Update cleaning status after completing tasks
- View assigned tasks and notifications
- Monitor their trust score and performance

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

The hardware requirements for the HYGO – Smart Hygiene Management System include IoT devices and computing components necessary for real-time hygiene monitoring. The system uses an ESP32 microcontroller for processing and communication, an MQ-135 gas sensor [6] to detect odour levels and an ultrasonic sensor to measure toilet usage. A stable Wi-Fi/internet connection is required for transmitting sensor data to the backend server. Additionally, a computer or mobile device is required to access the web-based dashboard for monitoring and management.

3.4.2 Software Requirements

HYGO is developed using a combination of software technologies to ensure efficient data processing, monitoring and analytics.The backend is implemented using Python with the Flask framework [8], which handles data processing, alert generation and communication between system components. The frontend dashboard is developed using HTML, CSS and JavaScript, providing a user-friendly interface for administrators and staff. The system uses a MySQL database [7] for secure storage of sensor data, cleaning logs, alerts and staff details. For analytics and prediction, machine learning libraries such as Scikit-learn [11], NumPy and Pandas are used, enabling models like Random Forest to predict usage patterns and cleaning requirements. The system is designed to run on web browsers, ensuring easy accessibility across devices.

3.4.3 Visual Studio Code

Visual Studio Code (VS Code) serves as the primary development environment for the HYGO system. It provides support for multiple programming languages such as Python, React, JavaScript and HTML, CSS along with powerful debugging tools and extensions. VS Code enhances productivity by offering features like code completion, error detection and integrated terminal support. The built-in Git integration allows developers to manage version control efficiently, track changes and collaborate effectively during development.

Chapter 4

SYSTEM DESIGN

4.1 Design Overview

The design of the HYGO – Smart Hygiene Management System focuses on modularity, scalability, reliability and maintainability. The system is structured such that each component performs a specific and well-defined function, ensuring clarity, efficient data flow and ease of future enhancement. The overall design integrates IoT hardware, backend services, database management, web-based interfaces and predictive analytics, allowing real-time hygiene monitoring and intelligent decision making.

The application follows a Layered Architecture combined with Model-View-Controller (MVC) principles to separate concerns and improve system maintainability. The overall system architecture is illustrated in Fig. 4.1, which shows the interaction between sensors, backend and user interface. The use case diagram (Fig. 4.2) represents the interaction between users and the system functionalities. The class diagram (Fig. 4.3) explains the structure of system components and their relationships. The activity diagram (Fig. 4.4) describes the workflow of system operations, while the data flow diagram (Fig. 4.5) illustrates the movement of data within the system. The database schema diagram (Fig. 4.6) and ER diagram (Fig. 4.7) represent the database structure and relationships among entities.

4.1.1 Design Approach

The system is divided into logical layers:

1. Presentation Layer (View)

Implementation:

- Web-based dashboard using HTML, CSS and JavaScript

Responsibilities:

- Displays real-time hygiene status and sensor readings
- Shows cleaning alerts and notifications
- Visualizes reports, analytics and predictions
- Allows administrators to monitor toilets and staff performance

This layer ensures a simple, responsive and user-friendly interface for all system users.

2. Application / Logic Layer (Controller)

Implementation:

- Backend application developed using Python with Flask

Responsibilities:

- Receives sensor data from IoT devices
- Performs threshold checks for hygiene conditions
- Generates cleaning alerts
- Manages staff monitoring and trust score calculation
- Coordinates predictive analytics and reporting
- Handles communication between the UI and database

This layer acts as the control unit of the system, managing overall application flow and decision-making.

3. Data Layer (Model)

Implementation:

- MySQL Database

Responsibilities:

- Stores real-time sensor readings
- Maintains toilet usage data
- Stores cleaning logs and staff records
- Stores trust scores and prediction results

This layer ensures persistent, structured and secure data storage for analysis and reporting.

4. Analytics and Prediction Layer

Implementation:

- AI/ML-based analytics models

Responsibilities:

- Analyze historical hygiene and usage data
- Predict toilet usage patterns
- Estimate cleaning frequency requirements
- Recommend optimal cleaning times

This layer performs the core intelligence and predictive functionality of the HYGO system.

4.1.2 System Architecture

The system architecture of HYGO defines the overall structure and interaction between hardware and software components to ensure efficient hygiene monitoring and management. It illustrates how IoT sensors, backend services, database and user interface work together to process real-time data and generate intelligent outputs. The complete architecture of the system is shown in Fig. 4.1.

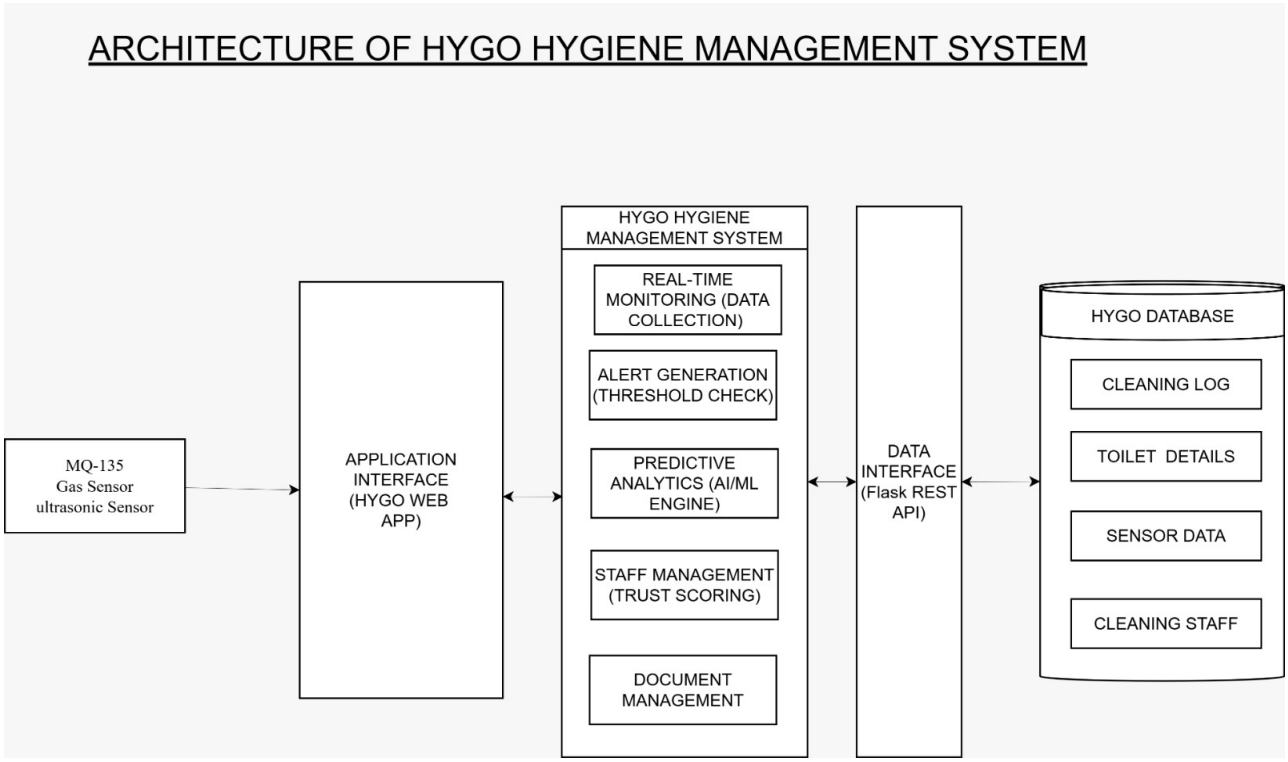

Figure 4.1: Visualization of system architecture

1.Front end

The Front End is the user interaction layer of the HYGO – Smart Hygiene Management System. It provides a web-based interface through which administrators and authorities can monitor real-time hygiene conditions, view cleaning alerts, track staff performance and access analytical reports. The front end displays live odour and usage data, alert notifications, trust scores and predictive insights in a clear and structured manner. Communication between the user interface and the backend server is managed through a RESTful API [10], which securely transfers data and system updates. This layer focuses on usability, responsiveness and clear visualization of hygiene status and system outputs to support quick and informed decision-making.

2.Back end

The Back End is the core processing and control layer of the HYGO system, where all data processing, alert generation and decision-making take place. Sensor data received from IoT devices is validated and processed in real time. The backend applies threshold-based logic to determine hygiene status and automatically generates cleaning alerts when required. This layer also manages staff monitoring, calculates trust scores based on cleaning response time, and coordinates predictive analytics using historical sensor and usage data [11]. In addition, the backend handles com- munication with the database, ensures secure data storage and sends processed results and alerts to the front end. Overall, the backend acts as the intelligence and decision-making unit of the HYGO system.

3.Database

The Database layer is responsible for storing and managing all data generated by the HYGO system. It maintains structured records of sensor readings, toilet usage data, cleaning logs, staff details, trust scores, alerts and prediction results. The database ensures efficient data retrieval and long-term storage through secure interaction with the backend [7]. Historical data stored in the database supports report generation, staff performance evaluation and predictive analytics. This layer provides reliable, secure and scalable data management, enabling effective hygiene monitoring and analysis within the HYGO system.

4.2 Module Design

The HYGO – Smart Hygiene Management System is divided into independent and well-defined modules to ensure modularity, scalability, maintainability and clarity in design [1]. Each module performs a specific function and interacts with other modules in a structured manner to support real time hygiene monitoring, automated alerts, staff tracking and predictive analytics.

4.2.1 Module Description

The system consists of the following major modules:

1. User Interface Module

Purpose:

- This module provides the web-based user interface for interacting with the HYGO system

Responsibilities:

- Display real-time odour and toilet usage data
- Show cleaning alerts and notifications
- Display staff performance and trust scores
- Visualize reports and predictive analytics
- Allow administrators to monitor hygiene status
- Provide system instructions and status messages

2. IoT Data Collection Module

Purpose:

- To collect real-time hygiene-related data from IoT sensors installed in public toilets

Responsibilities:

- Collect odour level data from gas sensors
- Collect toilet usage data from ultrasonic sensors
- Transmit sensor readings to the backend server
- Ensure continuous and reliable data flow

3. Data Processing and Alert Module

Purpose:

- To analyze incoming sensor data and determine hygiene conditions

Responsibilities:

- Validate and preprocess sensor readings
- Compare readings with predefined thresholds
- Generate automatic cleaning alerts when required
- Log alert events for future analysis

4. Staff Monitoring and Trust Score Module

Purpose:

- To track cleaning activities and evaluate staff performance

Responsibilities:

- Monitor cleaning response time
- Record cleaning activity logs
- Calculate and update staff trust scores
- Maintain performance history for evaluation

5. Database Management Module

Purpose:

- To manage secure storage and retrieval of system data

Responsibilities:

- Store sensor readings and usage data
- Store cleaning logs and staff records
- Store trust scores and alerts
- Store prediction results and reports
- Retrieve historical data when required
- Maintain database consistency and integrity

6. Prediction and Analytics Module

Purpose:

- To provide intelligent insights using historical data

Responsibilities:

- Analyze past sensor and usage data
- Predict toilet usage patterns
- Estimate cleaning frequency requirements
- Recommend optimal cleaning times

7. Application Control Module

Purpose:

- To coordinate communication and workflow between system modules

Responsibilities:

- Control overall system workflow
- Receive data from IoT modules
- Trigger data processing and alert modules
- Invoke prediction and analytics module
- Manage data storage operations
- Send processed data to the UI module

4.2.2 Module Description

The modules interact in a sequential and structured manner to complete the hygiene monitoring and management process. Step-by-Step Interaction Flow:

- The IoT Data Collection Module gathers odour and usage data from sensors
- The collected data is transmitted to the Application Control Module
- The data is passed to the Data Processing and Alert Module for analysis
- If hygiene thresholds are exceeded, cleaning alerts are generated
- Cleaning activities are tracked by the Staff Monitoring and Trust Score Module
- Historical data is sent to the Prediction and Analytics Module for insights
- All generated data is stored using the Database Management Module
- The processed results, alerts, reports and predictions are displayed through the User Interface Module

4.3 UML Diagram

4.3.1 Use Case Diagram

The use case of the HYGO – Smart Hygiene Management System defines the interaction between different users and the system functionalities. It illustrates how users such as administrators and cleaning staff interact with the system to perform various operations. The use case diagram is shown in Fig. 4.2.

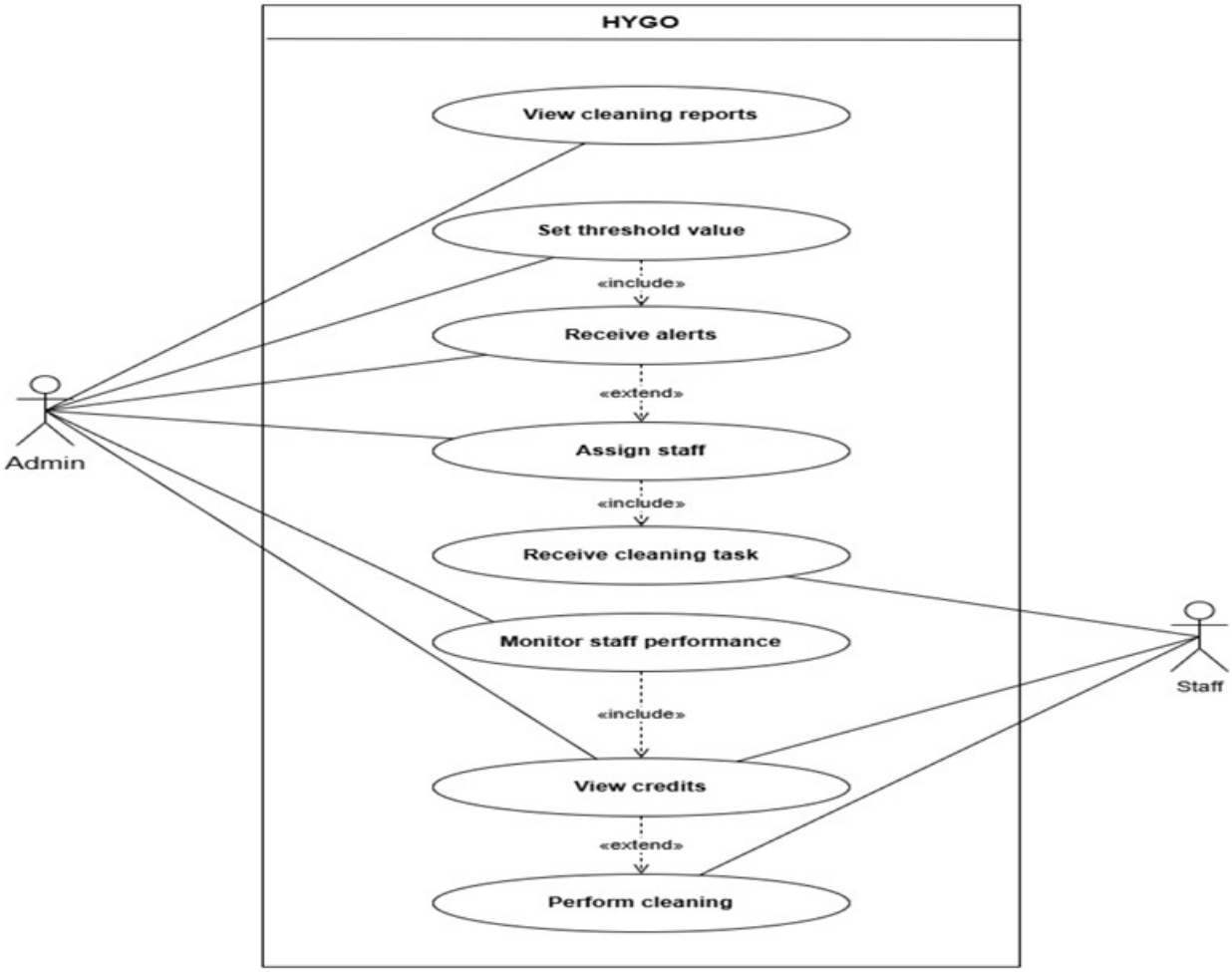

Figure 4.2: Visualization of Use case Diagram

User Roles and Responsibilities

- **Administrator (Admin)**
  - Monitor real-time hygiene conditions through the dashboard
  - Receive automatic cleaning alerts when thresholds are exceeded
  - Assign cleaning tasks to staff based on alerts
  - Set and update hygiene threshold values

- View cleaning reports and analytics
- Monitor staff performance and trust scores
- Manage complaints and update their status

- **Cleaning Staff (Staff)**
  - Receive cleaning alerts and assigned tasks from the system
  - Perform cleaning activities based on alerts
  - Update cleaning status after completing tasks
  - View assigned tasks and notifications
  - Monitor their trust score and performance

4.3.2 Class Diagram

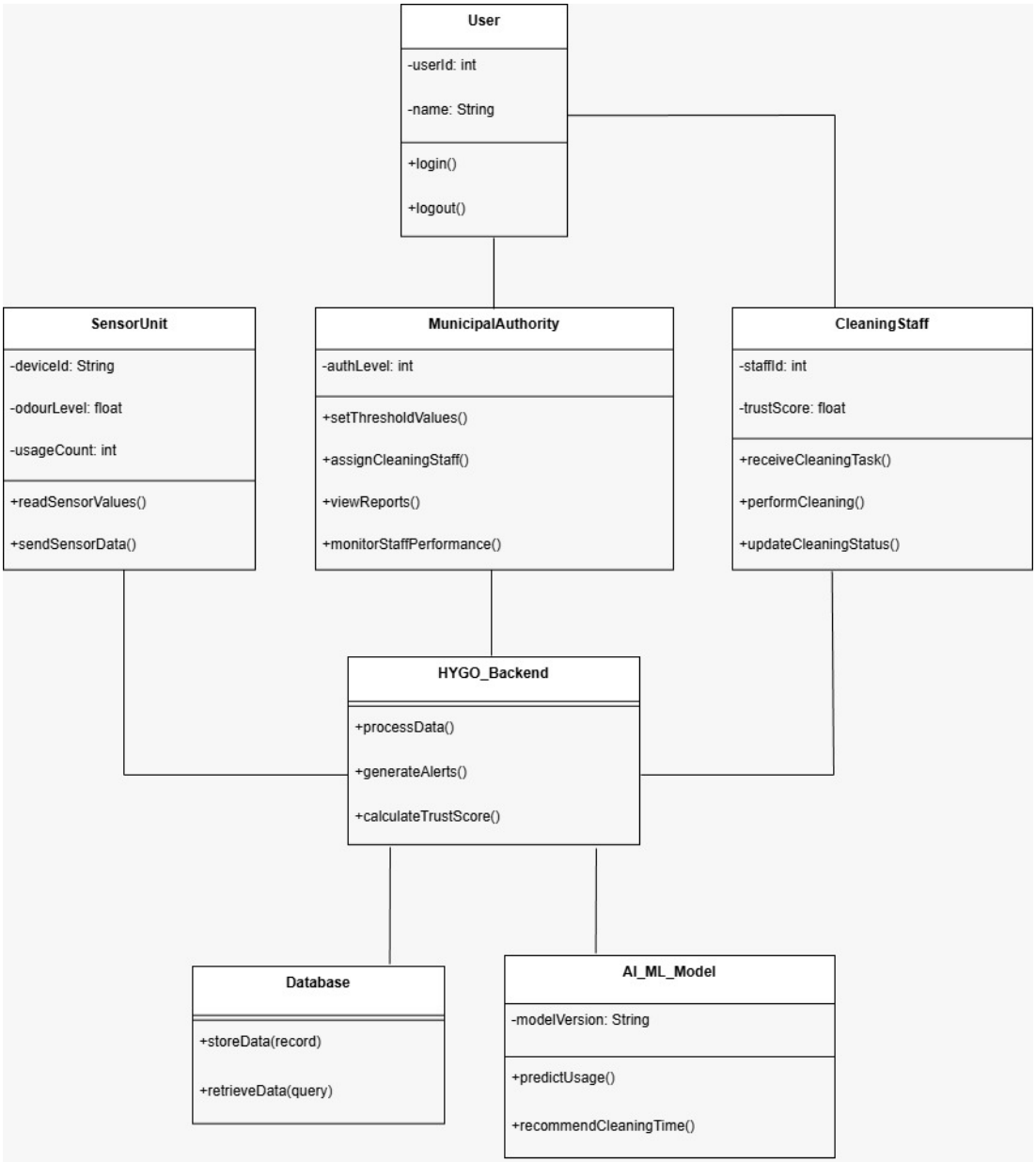

Figure 4.3: Visualization of Class Diagram

This diagram represents the object-oriented structure of the HYGO system, showing different classes, their attributes and functions, and how they interact with each other. Each class represents a major component of the system, and the methods define the operations performed. The class diagram of the system is illustrated in Fig. 4.3.

Main Classes Explanation

- User

  The User class is the base class representing all users in the system (Admin and Staff).

  Functions:

  - login() → Allows user to access the system
  - logout() → Ends the session securely

- Municipal Authority (Admin)

  This class represents the administrator who manages the entire system.

  Functions:

  - setThresholdValues() → Sets hygiene limits (odour/usage)
  - assignCleaningStaff() → Assigns cleaning tasks to staff
  - viewReports() → Views cleaning reports and analytics
  - monitorStaffPerformance() → Tracks staff efficiency and trust score

- Cleaning Staff

  This class represents the workers responsible for cleaning tasks.

  Functions:

  - receiveCleaningTask() → Receives assigned cleaning tasks
  - performCleaning() → Performs cleaning activity
  - updateCleaningStatus() → Updates task completion in system

- Sensor Unit

  This class represents the IoT hardware system.

  Functions:

  - readSensorValues() → Reads odour and usage data
  - sendSensorData() → Sends collected data to backend

- HYGO Backend

  This is the core processing unit of the system.

  Functions:

  - processData() → Processes sensor data
  - generateAlerts() → Generates cleaning alerts
  - calculateTrustScore() → Calculates staff performance score

- Database

  This class represents the data storage system.

  Functions:

  - storeData() → Saves sensor data, logs and alerts
  - retrieveData() → Fetches stored data when required

- AI/ML Model

  This class represents the intelligent prediction system.

  Functions:

  - predictUsage() → Predicts toilet usage patterns,Uses machine learning (Random Forest) for smart decisions.
  - recommendCleaningTime() → Suggests optimal cleaning time

4.3.3 **Activity Diagram**

The activity diagram of the HYGO – Smart Hygiene Management System represents the overall workflow of the system, illustrating how data flows from sensors to decision-making and cleaning actions. It describes the sequence of activities starting from real-time data collection using IoT sensors, followed by monitoring hygiene conditions, generating alerts when thresholds are exceeded and assigning cleaning tasks to staff. The diagram also shows how cleaning actions are recorded, performance is evaluated and data is used for further analysis and prediction. Overall, it provides a clear understanding of the system's dynamic behavior and operational flow. The complete workflow of the system is shown in Fig. 4.4.

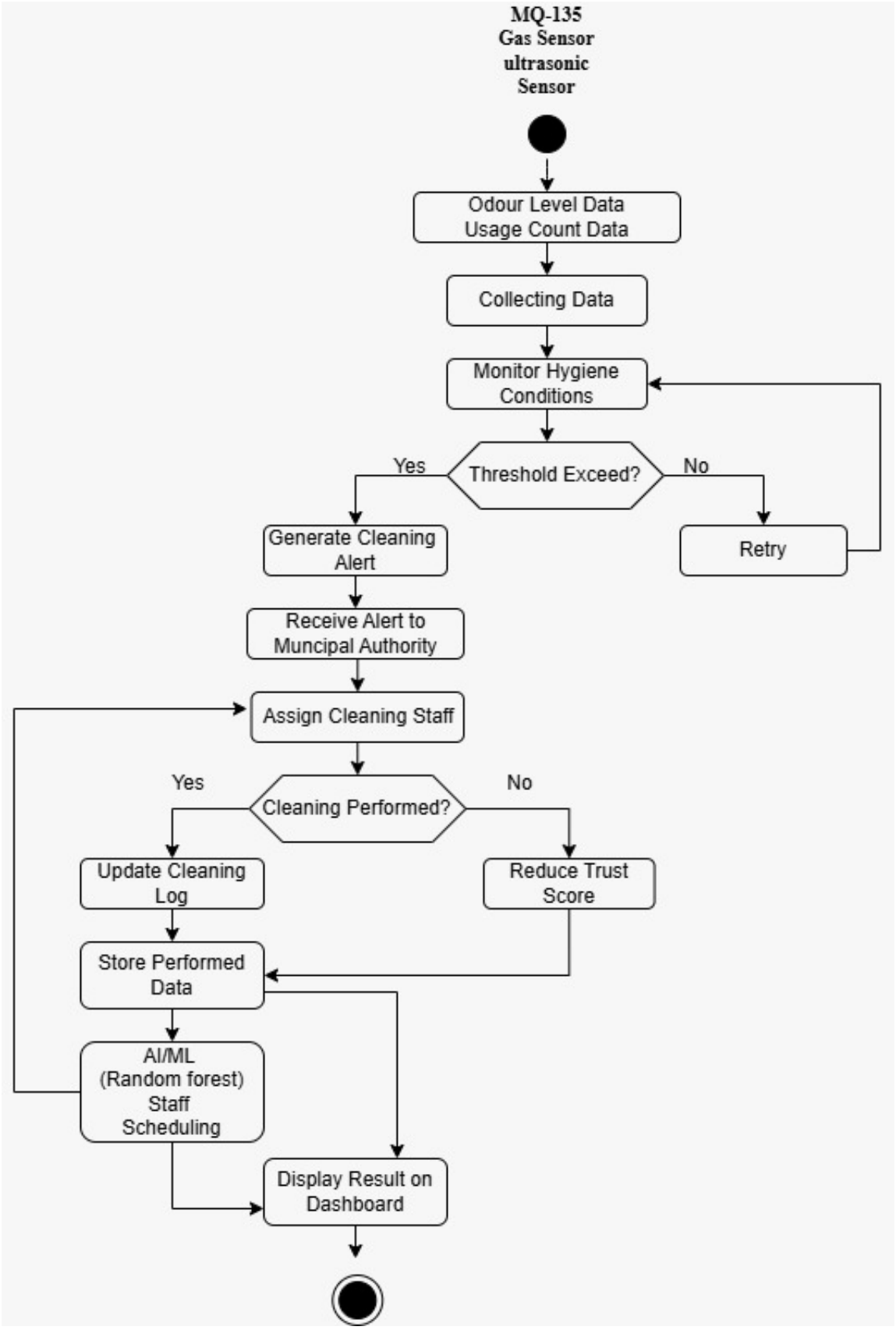

Figure 4.4: Visualization of Activity Diagram

Flow Explanation

- System starts by collecting data from MQ-135 (odour) and ultrasonic sensors (usage)
- Data is processed to monitor hygiene conditions

- System checks threshold value
- If NO → Retry and continue monitoring
- If YES → Generate cleaning alert
- Alert is sent to municipal authority (admin)
- Admin assigns cleaning staff
- System checks if cleaning is performed:
  - If YES → Update cleaning log and store performance data
  - If NO → Reduce trust score
- Data is sent to AI/ML model (Random Forest) for scheduling
- Final results are shown on dashboard

4.4 Dataflow Diagram

The data flow diagram (DFD) of the HYGO – Smart Hygiene Management System illustrates how data moves between different components of the system. It shows the flow of sensor data, processing in the backend, storage in the database and output to the user interface. The complete data flow of the system is represented in Fig. 4.5.

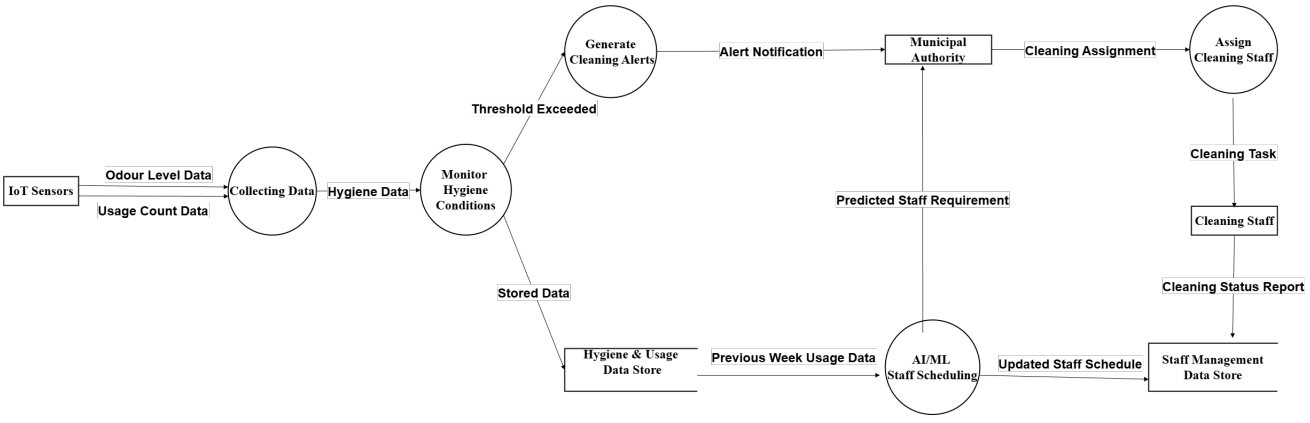

Figure 4.5: Visualization of Dataflow Diagram

System Flow

- IoT Sensors send odour and usage data
- Data is sent to the processing module
- System checks hygiene condition
- If threshold is exceeded, cleaning alerts are generated

- Alert is sent to municipal authority
- Authority assigns cleaning staff
- Cleaning activity updates the system
- Data is stored and used by AI/ML for prediction
- Output is displayed as reports and dashboard

4.5 Database Design

4.5.1 Database Schema

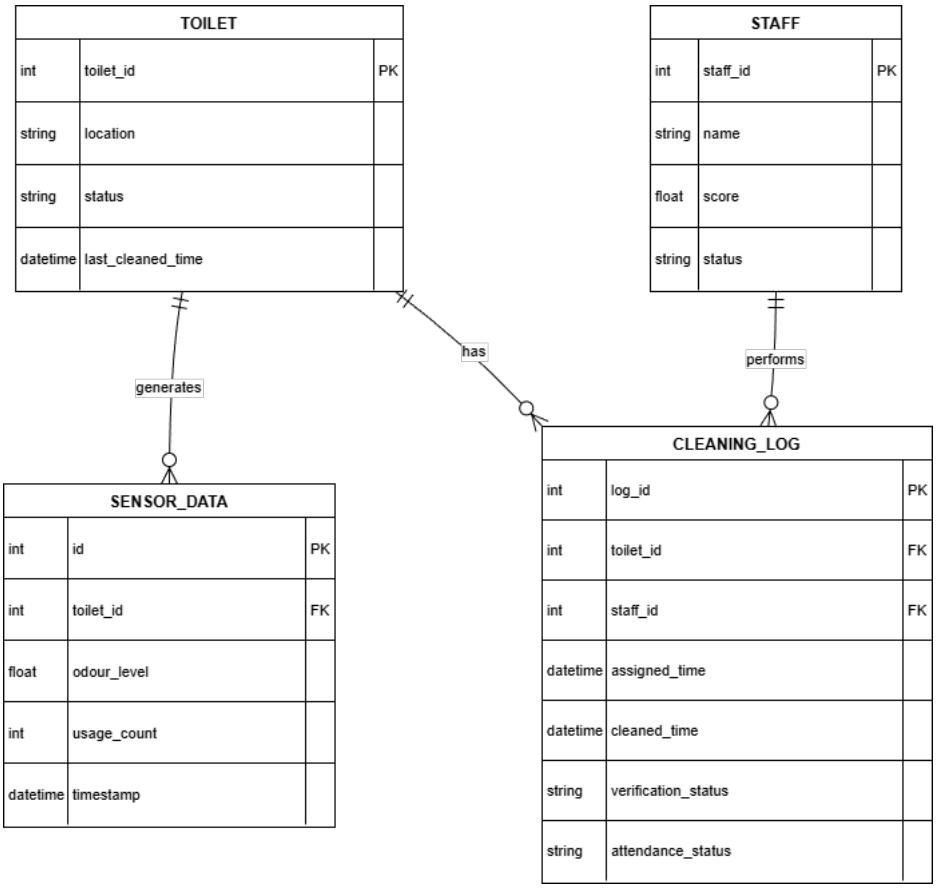

Figure 4.6: Visualization of Database Schema Diagram

The database schema of the HYGO – Smart Hygiene Management System is designed using a relational model to efficiently store and manage data related to hygiene monitoring, cleaning activities and system analytics. The system consists of multiple interconnected tables such as **Staff**, **Toilet**, **Sensor_Data**, **Cleaning_Log**, **Alerts**, **Complaints** and **Prediction**. The database schema of the system is illustrated in Fig. 4.6.

The **Staff** table stores details of cleaning staff including staff ID, name, status and trust score. The **Toilet** table maintains information about each toilet such as toilet ID, location, current status and last cleaned time. The **Sensor_Data** table records real-time data collected

from IoT sensors, including odour level, usage count and timestamp, and is linked to the **Toilet** table through a foreign key.

The **Cleaning_Log** table stores details of cleaning activities such as assigned time, cleaned time, verification status and attendance status, and is associated with both **Staff** and **Toilet** tables. The **Alerts** table stores automatically generated cleaning alerts along with their status and timestamp, while the **Complaints** table records user-reported hygiene issues and their resolution status.

Additionally, the **Prediction** table stores outputs from the AI/ML model, including predicted usage, cleaning frequency and recommended cleaning time.

All tables are connected using primary and foreign keys to maintain referential integrity, and constraints such as **ON DELETE CASCADE** ensure consistency by automatically removing related records when necessary. This structured schema enables efficient data storage, retrieval, monitoring and analysis within the HYGO system.

4.5.2 ER Diagram

The Entity Relationship (ER) diagram of the HYGO – Smart Hygiene Management System illustrates the logical structure of the database by representing entities, their attributes and the relationships between them. The diagram includes key entities such as Staff, Toilet, Sensor Data, Cleaning Log and Complaints, along with their associated attributes and relationships like cleans, generates and cleaned by. It clearly shows how data flows and is interconnected within the system, ensuring efficient storage, retrieval and management of hygiene-related information. The ER diagram also highlights the use of primary keys and foreign keys to maintain data integrity and consistency. The complete ER diagram of the system is shown in Fig. 4.7.

Design Entities

- Admin
- Staff
- Toilet
- Sensor Data
- Cleaning Log
- Alerts

- Staff Performance
- Complaints
- Prediction & Analytics

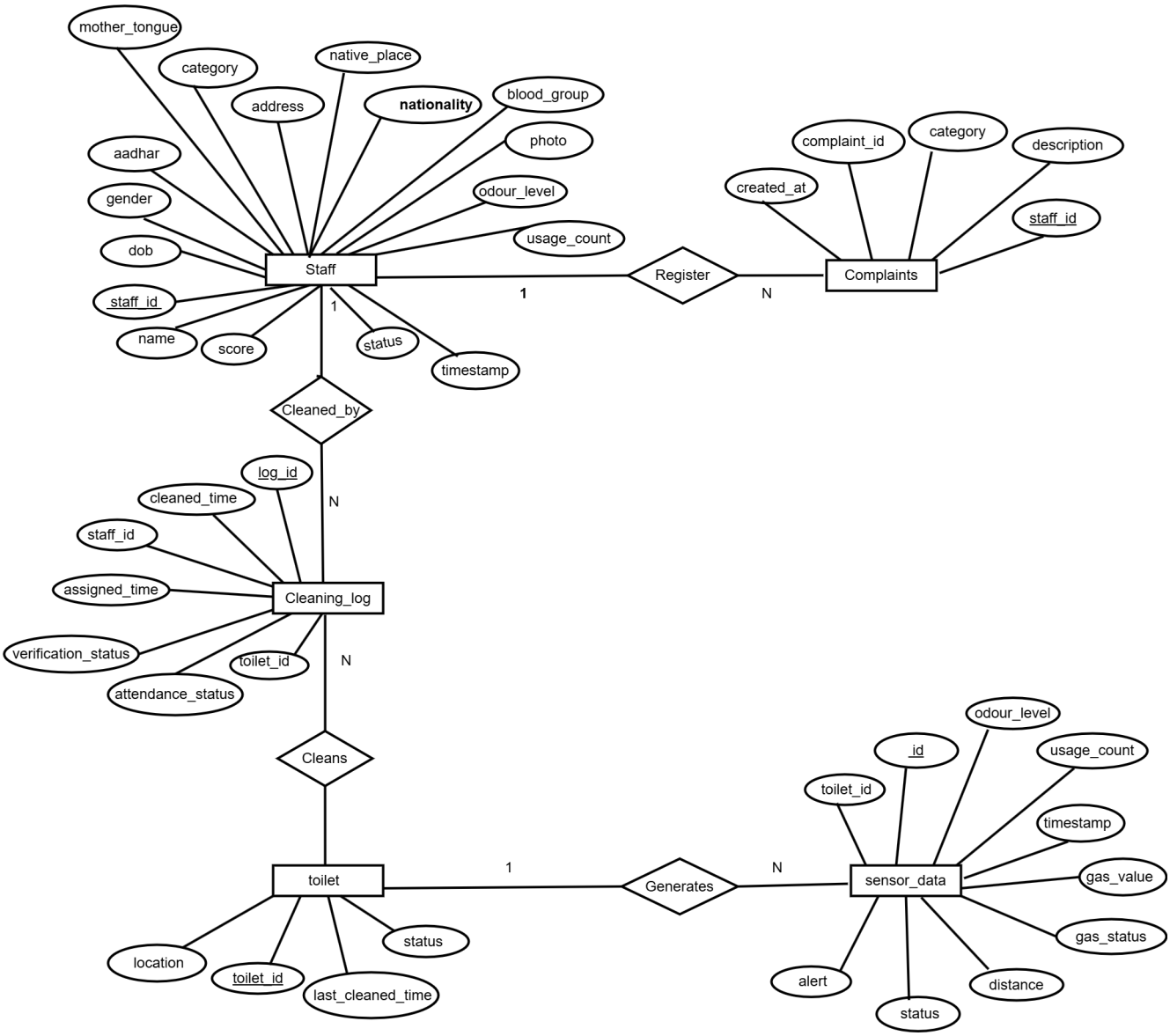

Figure 4.7: Visualization of ER Diagram

Design Attributes

Table Name	Description
Admin	Holds the information of system administrators who monitor and control the system
Staff	Holds the information of cleaning staff such as name, status and performance
Toilet	Stores details of toilets including location, status and last cleaned time
Sensor Data	Stores real-time data collected from sensors such as odour level and usage count
Cleaning Log	Stores records of cleaning activities including assigned time and cleaned time
Alerts	Stores information about automatic cleaning alerts generated by the system
Staff Performance	Stores trust score and performance details of cleaning staff
Complaints	Stores user complaints regarding hygiene issues
Prediction & Analytics	Stores predicted data such as usage patterns, cleaning frequency and best cleaning time

4.6 Interface Design

4.6.1 User Interface Design

- **Hygiene Monitoring Dashboard:** A centralized web-based dashboard that displays real-time odour levels, toilet usage data, hygiene status indicators and active cleaning alerts. This dashboard allows administrators to continuously monitor public toilet conditions from a single interface [13].
- **Sensor Data Visualization Panel:** A dedicated section that visually presents live sensor readings using charts and indicators, enabling quick understanding of hygiene conditions and usage trends [6].
- **Alert and Notification Panel:** An interface component that highlights automatic cleaning alerts, alert status and cleaning task assignments to ensure timely action by cleaning staff [5].
- **Staff Performance and Trust Score Panel:** A structured interface for viewing cleaning staff activity logs, response times and dynamically updated trust scores to support

performance evaluation and accountability [9].

- **Reports and Analytics Viewer:** A responsive interface for reviewing historical data, predictive analytics reports, usage trends, cleaning frequency predictions and best cleaning time recommendations [11].
- **Administrative Control Panel:** An interface for configuring system thresholds, managing staff records, monitoring overall system performance and maintaining hygiene monitoring settings.

4.6.2 Input And Output Design

- **Major Inputs:** Real-time odour sensor data, toilet usage count data, cleaning activity updates, staff details, historical hygiene data, system configuration parameters and predictive analytics models [6].
- **Major Outputs:** Automatic cleaning alerts, real-time hygiene status indicators, staff trust scores, cleaning logs, predictive insights (usage, cleaning frequency, best cleaning time), analytical reports and stored historical records in the database [7, 11].

Chapter 5

PROJECT WORK PLANNING

5.1 Gantt Chart

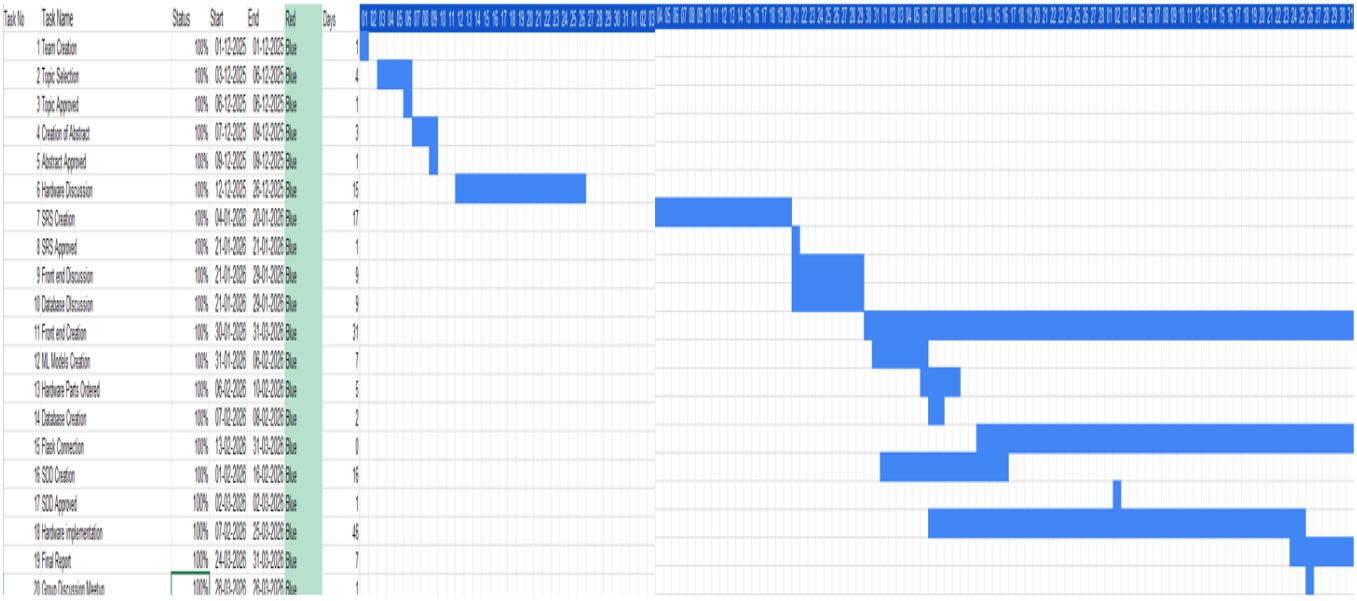

Figure 5.1: Visualization of Gantt Chart

Chapter 6

IMPLEMENTATION AND TESTING

6.1 Implementation

The HYGO – Smart Hygiene Management System is implemented as an IoT-based smart monitoring system integrated with a web-based dashboard. The system uses IoT sensors, a backend server developed using Python with Flask and a MySQL database for data storage and management. The application aims to ensure clean and hygienic public toilets by providing real-time monitoring, automated cleaning alerts, staff performance tracking and predictive analytics. The system ensures efficient monitoring and data-driven decision-making to maintain hygiene standards.

Upon deployment, authorized users such as administrators and cleaning staff can access the system through a web-based interface. Administrators can monitor hygiene conditions, assign cleaning tasks and analyze reports, while cleaning staff receive alerts and update cleaning activities. Users can then access the following core features:

- Real-Time Hygiene Monitoring: Continuously monitors odour levels and toilet usage using IoT sensors and displays live data on the dashboard.
- Automatic Cleaning Alerts: Generates alerts when hygiene conditions exceed predefined thresholds, ensuring timely cleaning actions.
- IoT-Based Data Collection: Uses sensors such as MQ-135 (odour detection) and ultrasonic sensors (usage detection) to collect real-time data.
- Staff Monitoring and Trust Score: Tracks cleaning activities and calculates trust scores based on response time and performance.
- Predictive Analytics: Analyzes historical data to predict toilet usage, cleaning frequency and the best time for cleaning.
- Dashboard Visualization: Provides a centralized interface to view real-time data, alerts, reports and analytics.

- Database Management: Stores sensor data, cleaning logs, staff details, alerts and prediction results securely in a MySQL database.
- Application Control and Data Processing: Processes incoming sensor data, validates conditions and coordinates communication between modules.

All system data and user interactions are securely stored in the database, ensuring real-time updates, efficient monitoring and reliable hygiene management.

6.1.1 Algorithms Used

A.1 Algorithm: System Initialization & Hardware Logic (ESP32)

Algorithm: Smart Toilet Monitoring and Alert System

Input:

- Gas sensor value (gasValue)
- Ultrasonic sensor distance (distance — averaged over 5 readings)

Output:

- Toilet hygiene status (CLEAN / MODERATE / DIRTY)
- Gas level label (LOW / MEDIUM / HIGH / CRITICAL)
- Usage count
- Alert status (ON / OFF)
- Data sent to backend via HTTP POST

Steps:

1. Start
2. Initialize:
   2.1. $SetisOccupied = \text{false}$
   2.2. $SetusageCount = 0$
   2.3. $SetalertTriggered = \text{false}$
   2.4. $SetwasGasHigh = \text{false}$
   2.5. $SetlastDistance = 100$
   2.6. Connect to WiFi (wait until $WL\_CONNECTED$)
3. Read Gas Sensor: Read analog value from GPIO 34 $\rightarrow gasValue$
4. Classify Gas Level:

4.1. If $gasValue < 180 \rightarrow$ LOW

4.2. Else if $gasValue < 250 \rightarrow$ MEDIUM

4.3. Else if $gasValue < 450 \rightarrow$ HIGH

4.4. Else $\rightarrow$ CRITICAL

5. Measure Distance (Averaged):

5.1. Repeat 5 times:

5.1.1. Send TRIG pulse ($10\mu s$)

5.1.2. Measure ECHO duration

5.1.3. Calculate $d = \frac{duration \times 0.034}{2}$

5.1.4. If $2 \leq d \leq 400 \rightarrow$ add to total, increment valid count

5.2. If $valid = 0 \rightarrow$ return $lastDistance$

5.3. Else $\rightarrow$ return $\frac{total}{valid}$

6. Occupancy Detection:

6.1. If $distance < 25 \wedge isOccupied =$ false:

6.1.1. $isOccupied \leftarrow$ true

6.1.2. $enterTime \leftarrow millis()$

6.1.3. Print ”ENTERED”

6.2. Else if $isOccupied =$ true AND:

6.2.1. $(currentTime - enterTime) \geq 15s$ OR

6.2.2. $(distance > 40)$

6.2.3. Then:

A. $isOccupied \leftarrow$ false

B. $usageCount \leftarrow usageCount + 1$

C. Print ”LEFT, COUNT: usageCount”

6.3. $lastDistance \leftarrow distance$

7. Gas-Based Cleaning Detection:

7.1. If $gasValue \geq 250$:

7.1.1. $wasGasHigh \leftarrow$ true

7.2. If $wasGasHigh =$ true $\wedge\, gasValue < 120$:

7.2.1. $usageCount \leftarrow 0$

7.2.2. $wasGasHigh \leftarrow$ false

7.2.3. Print ”CLEANED $\rightarrow$ COUNT RESET”

8. Alert Logic:
   8.1. If $gasValue \geq 250 \vee usageCount \geq 30$:
      8.1.1. $alertTriggered \leftarrow$ true
   8.2. Else:
      8.2.1. $alertTriggered \leftarrow$ false
9. Hygiene Status Classification:
   9.1. If $gasValue \geq 250 \vee usageCount \geq 30 \rightarrow$ status = ”DIRTY”
   9.2. Else if $gasValue \geq 120 \vee usageCount \geq 15 \rightarrow$ status = ”MODERATE”
   9.3. Else $\rightarrow$ status = ”CLEAN”
10. Prepare JSON Payload: Create JSON with toilet_id, gas_value, gas_status, distance, status, count, alert
11. Send Data:
   11.1. Check WiFi status == $WL\_CONNECTED$
   11.2. If connected $\rightarrow$ HTTP POST JSON to backend URL
   11.3. If not connected $\rightarrow$ print warning, skip send
12. Wait: Delay $2000\,ms$
13. Repeat Loop: Go to Step 3
14. End

A.2 Algorithm: Sensor Data Ingestion (/api/data)

First backend algorithm — receives ESP32 data

Input: ESP32 JSON payload

Output: Data stored in MySQL sensor_data table

Steps:

1. Start
2. Receive ESP32 JSON (gas_value, gas_status, distance, status, count, alert)
3. Extract values from JSON
4. Convert gas to odour level: odour_level = gas_value / 100
5. Normalize status to lowercase
6. Insert into sensor_data table with toilet_id, odour_level, usage_count, gas_value, gas_status, distance, status, alert, timestamp = NOW()

7. Commit changes
8. Return success message
9. End

A.3 Algorithm: Login API (/api/login)

User authentication before accessing dashboard

Input: username, password

Output:role (admin / staff), staff_id if staff

Steps:

1. Start
2. Receive JSON (username, password)
3. If data is missing → return 400 error
4. Connect to database
5. Check users table: If match → return role = ”admin”
6. Else check staff table: If match → return role = ”staff” + staff_id
7. Else → return 401 invalid credentials
8. Close database connection
9. End

A.4 Algorithm: Toilets API (/api/toilets)

Displays real-time toilet status on dashboard

Input: Latest sensor_data per toilet

Output: Toilet status list (clean / moderate / dirty / needs cleaning)

Steps:

1. Fetch toilets joined with latest sensor data using correlated subquery MAX(timestamp)
2. Determine status using FSM:
   2.1. If odour $>$ 2.5 OR status = ’dirty’ → ”dirty”
   2.2. Else if odour $>$ 1.2 OR status = ’moderate’ → ”moderate”
   2.3. Else if last_cleaned $>$ 6 hours ago → ”needs cleaning”
   2.4. Else if sensor data exists → ”clean”
   2.5. Else → ”online”

3. Return toilet list with status, odour_level, gas_value, distance
4. End

POST:

Steps:

1. Receive toilet_id, location, status
2. Validate required fields
3. Insert into toilet table with last_cleaned_time = NOW()
4. Return success
5. End

A.5 Algorithm: Cleaning Alerts (/api/cleaning-alerts)

Generates prioritized cleaning alerts for admin

Input: Latest sensor data + last_cleaned_time per toilet

Output: Prioritized alert list

Steps:

1. Start
2. Fetch latest sensor reading per toilet
3. For each toilet classify alert:
   3.1. If odour $>$ 2.5 AND usage $>$ 30 $\rightarrow$ URGENT_CLEANING
   3.2. Else if odour $>$ 2.5 $\rightarrow$ HIGH_ODOUR
   3.3. Else if odour $>$ 1.2 $\rightarrow$ MODERATE_ODOUR
   3.4. Else if usage $>$ 30 $\rightarrow$ HIGH_USAGE
4. Separately check last_cleaned_time: If NULL OR last_cleaned $>$ 6 hours ago AND no sensor alert $\rightarrow$ NOT_CLEANED
5. Use alerts_dict keyed by toilet_id to prevent duplicates
6. Sensor alerts take priority over time-based alerts
7. Return alert list
8. End

A.6 Algorithm: Alerts Management (/api/alerts)

Displays severity-based alerts in incidents tab

Input: Latest sensor_data per toilet

Output: Alert list with severity levels

Steps:

1. Start
2. Fetch latest sensor data per toilet using MAX(timestamp)
3. Classify severity:
   3.1. If odour $>$ 2.5 $\rightarrow$ ”critical”
   3.2. Else if usage $>$ 30 $\rightarrow$ ”medium”
   3.3. Else $\rightarrow$ ”low”
4. Filter: only return toilets where odour $>$ 1.2 OR usage $>$ 30
5. Return alerts with id, severity, status, message, time
6. End

A.7 Algorithm: Assign Staff (/api/alerts/¡id¿/assign)

Admin assigns a staff member to clean a toilet

Input: alert_id, staff_id

Output: Cleaning log entry created

Steps:

1. Start
2. Receive alert_id (e.g. T-10) and staff_id
3. Validate staff_id is provided
4. Strip ”T-” prefix from alert_id $\rightarrow$ extract toilet_id
5. Insert into cleaning_log:
   5.1. toilet_id, staff_id, assigned_time = NOW()
   5.2. attendance_status = ”Assigned”
   5.3. verification_status = ”Pending”
6. Commit to database
7. Return success
8. End

A.8 Algorithm: Staff Tasks (/api/staff/assigned-tasks/¡staff_id¿)

Staff views their assigned cleaning tasks

Input: staff_id

Output: List of assigned tasks

Steps:

1. Receive staff_id from URL
2. Fetch from cleaning_log where:
   2.1. staff_id matches
   2.2. attendance_status = ”Assigned”
3. Return tasks with log_id, toilet_id, assigned_time, priority
4. End

A.9 Algorithm: AI Prediction (/api/predict-from-db)

Predicts cleaning priority for each toilet using ML model

Input: Latest sensor data + cleaning history per toilet

Output: Predicted priority score, status, cleanliness score per toilet

Steps:

1. Start
2. Fetch latest sensor data and last cleaned time per toilet
3. For each toilet:
   3.1. If no sensor data $\rightarrow$ return status = ”data pending”, score = 0
   3.2. If never cleaned $\rightarrow$ set $last\_cleaned = 24$ hours ago
   3.3. Calculate $hours\_since\_clean$
   3.4. Prepare features: $[usage\_count, time\_of\_day, hours\_since\_clean]$
   3.5. Predict: $priority\_score = model.predict(features)$
   3.6. Classify status:
      3.6.1. If $odour > 2.5 \vee sensor =$ dirty $\rightarrow$ status = dirty, score = 30
      3.6.2. Else if $odour > 1.2 \vee sensor =$ moderate $\rightarrow$ status = moderate, score = 60
      3.6.3. Else $\rightarrow$ status = clean, score = 90
4. Return predictions list with $toilet\_id$, $predicted\_minutes$, $status$, $usage\_count$, $cleanliness\_score$
5. End

A.10 Algorithm: AI Model Training

Algorithm: Hygiene Priority Prediction using Machine Learning

Input: Dataset (hygo_ml.csv)

Output: Trained model (hygo_model.pkl)

Steps:

Phase 1 — Data Preprocessing:

1. Start
2. Load dataset using pandas
3. Convert timestamp to datetime format
4. Extract features: hour and weekday from timestamp
5. Drop unnecessary columns: timestamp, id

Phase 2 — Feature Engineering:

6. Compute target variable: priority_score = (usage_count $\times$ 0.6) + (odour_level $\times$ 40)
7. Select input features: toilet_id, usage_count, odour_level, hour, weekday

Phase 3 — Model Training:

8. Split dataset: 80% training, 20% testing
9. Initialize Random Forest Regressor (n_estimators = 100)
10. Train model: model.fit(X_train, y_train)

Phase 4 — Model Evaluation:

11. Predict on test data: predictions = model.predict(X_test)
12. Calculate MAE (Mean Absolute Error)
13. Calculate R^2 score

Phase 5 — Model Saving:

14. Save model: joblib.dump(model, ”hygo_model.pkl”)
15. End

A.11 Algorithm: Reports (/api/reports)

Generates analytics for admin dashboard

Input: cleaning_log table data

Output: Report metrics and charts

Steps:

1. Count total completed cleanings WHERE cleaned_time IS NOT NULL
2. Compute average cleaning duration: AVG(TIMESTAMPDIFF(MINUTE, assigned_time, cleaned_time))
3. Calculate verification rate: SUM(Verified) / COUNT(*) $\times$ 100
4. Fetch daily cleaning counts GROUP BY DATE(cleaned_time)
5. Return total_cleanings, avg_duration, verification_rate, daily_activity
6. End

A.12 Algorithm: Feedback API (/api/feedback)

Input: cleaning_log with cleaned_time

Output: Feedback list

Steps:

1. Fetch all cleaning_log records where cleaned_time IS NOT NULL
2. Format response with toilet_id, verification_status, staff_id, cleaned_time
3. Return sorted by latest cleaned_time
4. End

A.13 Algorithm: Complaints (/api/complaints)

POST:

Steps:

1. Receive category, description, staff_id
2. Insert into complaints table
3. Commit and return success
4. End

Steps:

1. Fetch all complaints from complaints table
2. Sort by created_at DESC (latest first)
3. Return complaint list
4. End

6.1.2 **Database Structures**

MySQL is a relational database management system used to store and manage structured data in the HYGO – Smart Hygiene Management System. The database organizes data into tables, where each table consists of rows representing records and columns representing attributes. Each table has a primary key to uniquely identify records, and relationships between tables are maintained using foreign keys to ensure data integrity and consistency.

In the HYGO system, multiple tables are used to store different types of data such as sensor readings, cleaning logs, staff details, alerts and user complaints. The main tables include Users, Staff, Toilet, SensorData, CleaningLogs, Alerts and Complaints.

The tables are structured as follows:

- Users Table
  - id: Unique identifier (Auto-incremented Primary Key)
  - username: Unique username for login
  - password: Stores user password securely (hashed)
  - role: Defines user type (Admin / Cleaning Staff)
  - staff_id: Foreign key linking to Staff table
- Staff Table
  - staff_id: Unique identifier (Auto-incremented Primary Key)
  - name: Name of the staff
  - score: Trust score (default value 100)
  - status: Active/Inactive status
  - phone: Contact number
  - address: Staff address
- Toilet Table
  - toilet_id: Unique identifier (Primary Key)
  - location: Location of the toilet
  - status: Current hygiene status (Clean/Dirty)

    - last_cleaned_time: Timestamp of last cleaning

- Sensor_Data Table
    - id: Unique identifier (Primary Key)
    - toilet_id: Foreign key linking to Toilet table
    - gas_value: Raw value from MQ-135 sensor
    - gas_status: Gas level (LOW / MEDIUM / HIGH / CRITICAL)
    - odour_level: Measured odour intensity
    - distance: Distance from ultrasonic sensor
    - status: Occupancy status (OCCUPIED / FREE)
    - usage_count: Number of times the toilet is used
    - alert: Alert status (ON / OFF)
    - timestamp: Time of data collection
- Alerts Table
    - id: Unique identifier (Primary Key)
    - sensorID: Foreign key referencing SensorData
    - alertStatus: Status of alert (Active / Resolved)
    - timestamp: Time when alert was generated
- Cleaning_Logs Table
    - log_id: Unique identifier (Primary Key)
    - toilet_id: Foreign key linking to Toilet table
    - staff_id: Foreign key linking to Staff table
    - assigned_time: Time when cleaning task was assigned
    - cleaned_time: Time when cleaning was completed
    - verification_status: Verification of cleaning
    - attendance_status: Staff attendance status
- Complaints Table
    - complaint_id: Unique identifier (Primary Key)
    - category: Type of complaint
    - description: Details of the complaint
    - staff_id: Foreign key linking to Staff table (optional)
    - created_at: Timestamp when complaint was submitted

The staff_id in the Users, CleaningLogs and Complaints tables links data to a specific staff member. The toilet_id in the SensorData and CleaningLogs tables connects records to a particular toilet. The sensorID field in the Alerts table connects alerts with sensor readings.

To maintain data integrity, foreign key constraints are enforced. The ON DELETE CASCADE constraint ensures that related records such as cleaning logs and complaints are automatically removed when a related record is deleted.

6.2 Testing

Software testing is the process of evaluating the HYGO – Smart Hygiene Management System to ensure that all functionalities such as real-time hygiene monitoring, automatic alert generation, staff tracking and predictive analytics work correctly. The main goal of testing is to identify errors, ensure system reliability and verify that the system meets user requirements. Testing also ensures that the system performs efficiently under real-time conditions and provides accurate results.

6.2.1 Test Case

A test case is a structured document that defines input data, execution conditions and expected outcomes to verify a specific functionality of the system. In HYGO, test cases are used to validate features like sensor data processing, alert generation, staff monitoring and dashboard updates. Each test case ensures that the system behaves correctly for both valid and invalid inputs.

6.2.2 Unit Testing

Unit testing is performed to test individual components of the HYGO system separately. Each module such as IoT data collection, alert generation, database operations and trust score calculation is tested independently to ensure correctness. This helps in identifying errors at an early stage before integrating the modules.

6.2.3 Integration Testing

Integration testing checks whether different modules of the HYGO system work together properly. It verifies the interaction between IoT sensors, backend server, database and dashboard. For example, it ensures that sensor data is correctly transmitted, processed and displayed on the dashboard without any data loss or delay.

6.2.4 Validation Testing

Validation testing ensures that the HYGO system meets real-world requirements and user expectations. It verifies whether the system effectively monitors hygiene conditions, generates alerts at the right time and helps administrators make informed decisions. This testing confirms that the system fulfills its intended purpose in real-time environments.

6.2.5 Black Box Testing

Black box testing is used to evaluate the system based on inputs and outputs without considering internal code structure. In HYGO, this includes checking whether alerts are generated correctly when odour levels exceed thresholds, whether dashboard displays correct data and whether users can interact with the system properly.

6.2.6 White Box Testing

White box testing focuses on the internal logic and code structure of the system. It verifies algorithms such as threshold-based alert generation, trust score calculation and prediction models. This ensures that all logical conditions, loops and data processing steps are functioning correctly within the system.

Test Case for Hardware (IoT Sensors)

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	ESP32 Wi-Fi connectivity	Valid SSID & password	ESP32 connects to Wi-Fi	Connected successfully	Success
2	MQ-135 sensor reading	Odour present	Odour value detected correctly	Detected correctly	Success
3	Ultrasonic sensor detection	User presence	Usage count increases	Count increased	Success
4	Data transmission	Sensor data	Data sent to server	Data received	Success
5	Real-time update	Continuous data	Data updates on dashboard	Updated correctly	Success
6	Sensor failure handling	No input	System handles error safely	Handled properly	Success

Test Case for Authentication (Login System)

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	User registration	Valid details	Account created successfully	Created	Success
2	User login	Valid credentials	Login successful	Logged in	Success
3	Invalid login	Wrong password	Error message displayed	Error shown	Success
4	Empty fields	No input	Login prevented	Prevented	Success

Test Case for Administrator (Admin Module)

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Admin login	Valid credentials	Admin logged in	Logged in	Success
2	Invalid login	Wrong credentials	Error message	Error shown	Success
3	View dashboard	Admin login	Dashboard displayed	Displayed	Success
4	Monitor hygiene	Sensor data	Data displayed correctly	Correct	Success
5	View alerts	Active alerts	Alerts displayed	Displayed	Success
6	Assign staff	Staff + alert	Task assigned	Assigned	Success
7	View performance	Staff data	Trust score shown	Displayed	Success
8	Update alert	Mark resolved	Status updated	Updated	Success
9	View reports	Historical data	Reports generated	Generated	Success
10	Configure threshold	New limit	Updated	Updated	Success
11	View complaints	Complaint data	Displayed	Displayed	Success
12	Update complaint	Change status	Updated	Updated	Success
13	Logout	Click logout	Logged out	Logged out	Success

Test Case for Dashboard (Hygiene Monitoring)

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Load dashboard	Valid login	Dashboard displayed	Displayed	Success
2	View odour level	Sensor data	Correct value shown	Correct	Success
3	View usage count	Sensor data	Usage displayed	Displayed	Success
4	Real-time updates	Continuous data	Live updates shown	Updated	Success
5	No data case	No input	Show default message	Displayed	Success

Test Case for Alert System

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Generate alert	High odour	Alert triggered	Triggered	Success
2	Normal condition	Low odour	No alert	No alert	Success
3	Alert display	Active alert	Alert visible on dashboard	Visible	Success
4	Resolve alert	Cleaning done	Alert removed	Removed	Success

Test Case for Staff Monitoring

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Record cleaning	Cleaning done	Log stored	Stored	Success
2	Trust score update	Delay in cleaning	Score reduced	Reduced	Success
3	Fast response	Quick cleaning	Score maintained	Maintained	Success

Test Case for Prediction Module (AI/ML)

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Usage prediction	Historical data	Predicted correctly	Correct	Success
2	Cleaning frequency	Past logs	Estimated correctly	Correct	Success

3	Best cleaning time	Data trends	Suggested time shown	Shown	Success

Test Case for Complaints Module

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Submit complaint	Valid data	Complaint stored	Stored	Success
2	View complaint	Existing data	Complaint displayed	Displayed	Success
3	Update status	Change status	Status updated	Updated	Success
4	No complaint	No data	Show message	Displayed	Success

Test Case for Database

No	Test Case	Input Data	Expected Result	Actual Result	Remark
1	Store sensor data	Real-time data	Data stored correctly	Stored	Success
2	Retrieve data	Query request	Correct data fetched	Fetched	Success
3	Data consistency	Compare UI & DB	Values match	Match	Success
4	Delete user	User removed	Related data deleted	Deleted	Success

6.3 Screenshots

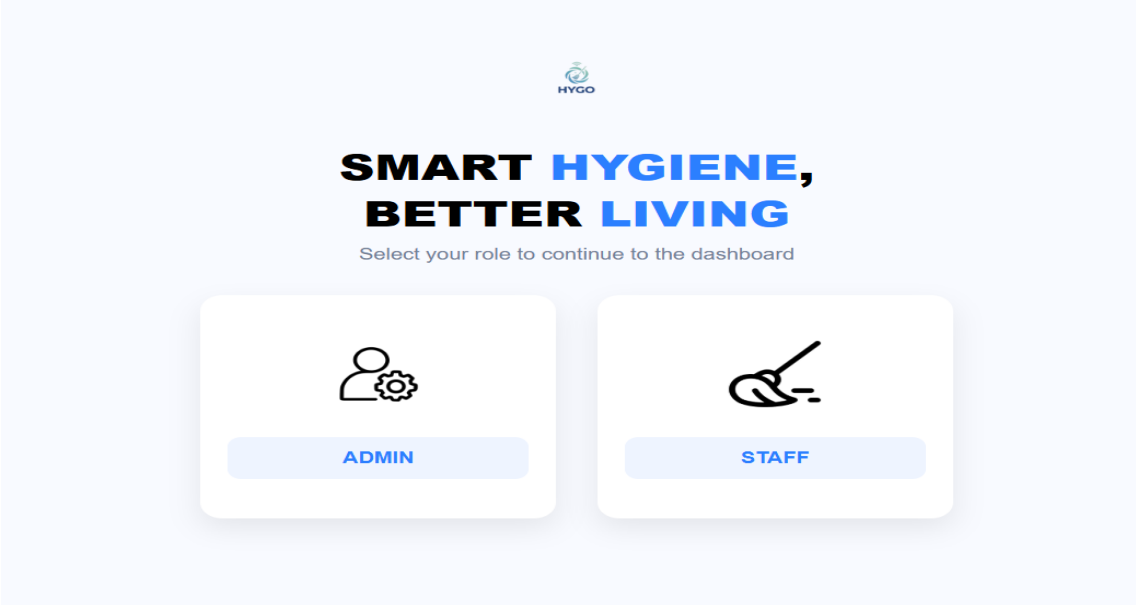

Figure 6.1: Home Page

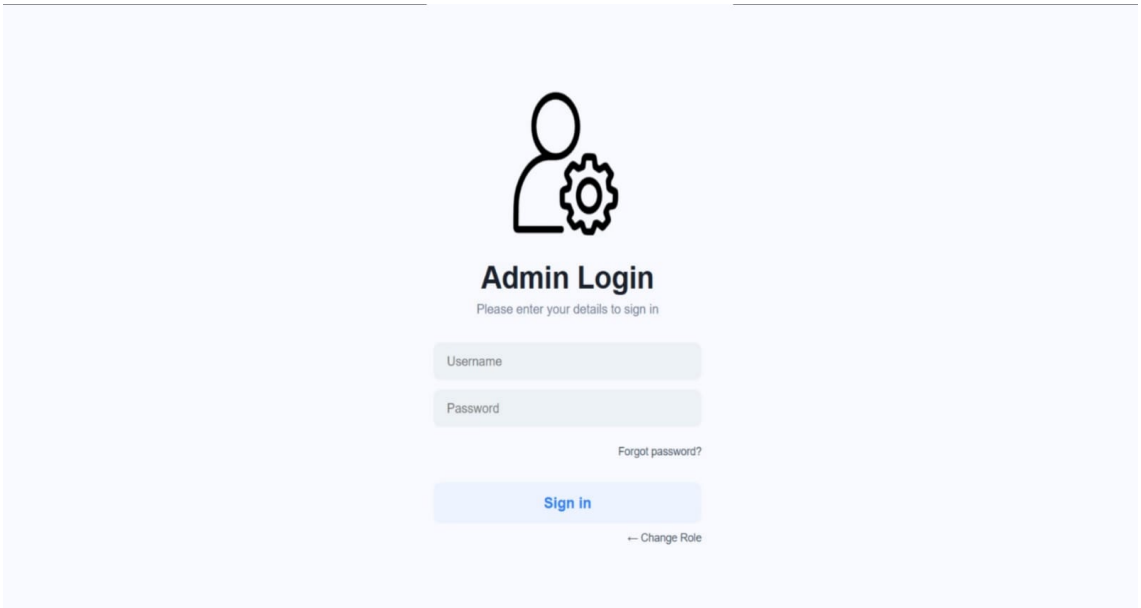

Figure 6.2: Admin Login

Figure 6.3: Admin Dashboard

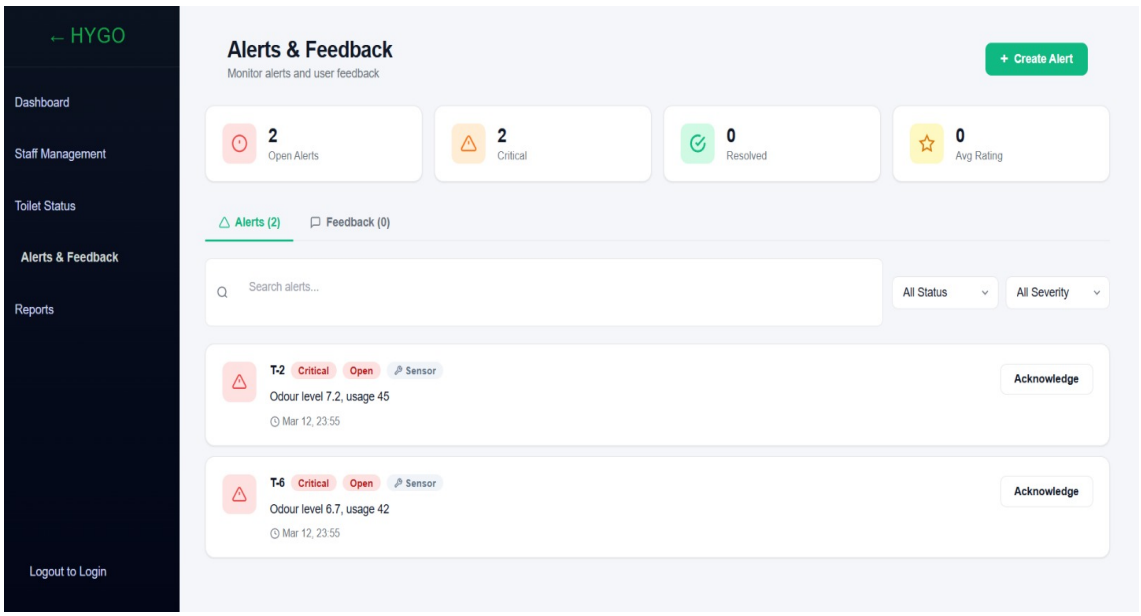

Figure 6.4: Active Alert

Figure 6.5: AI Cleaning Prediction

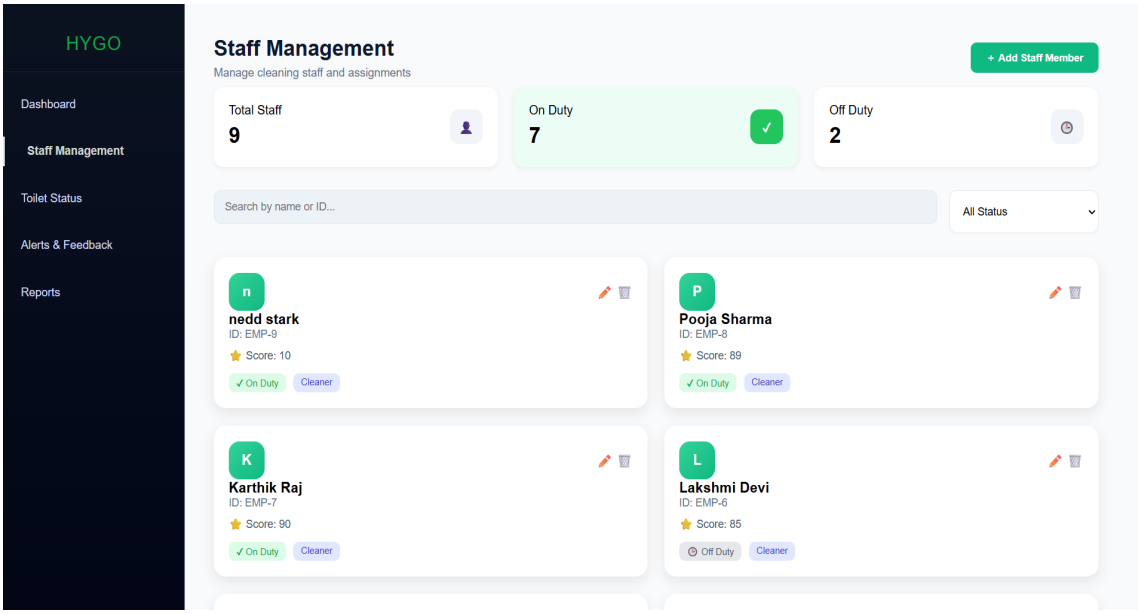

Figure 6.6: Staff Management

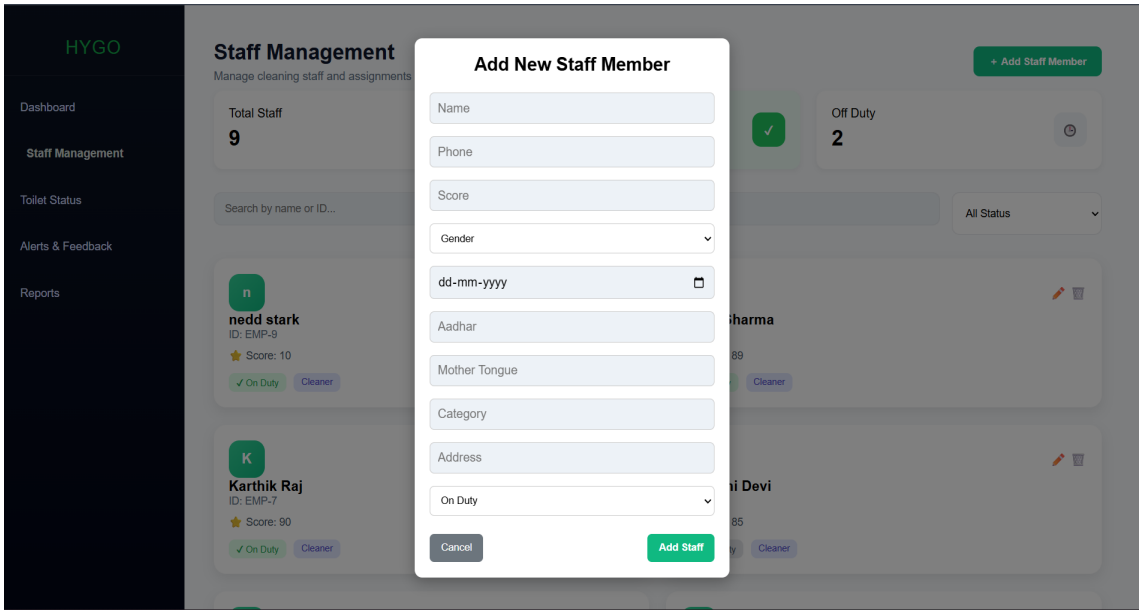

Figure 6.7: Add Staff Members

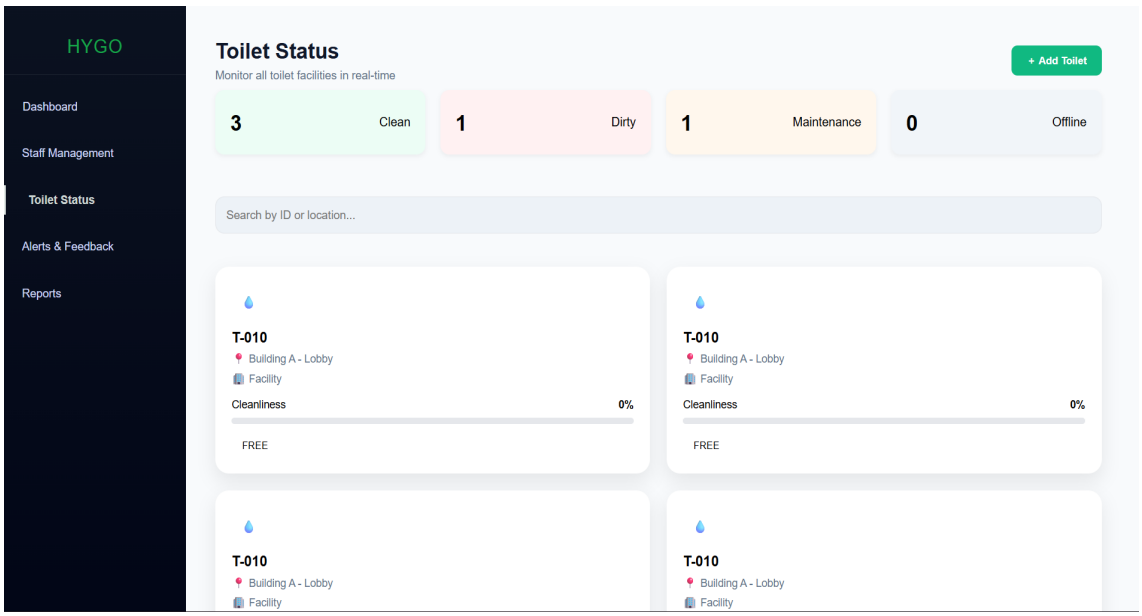

Figure 6.8: Toilet Status

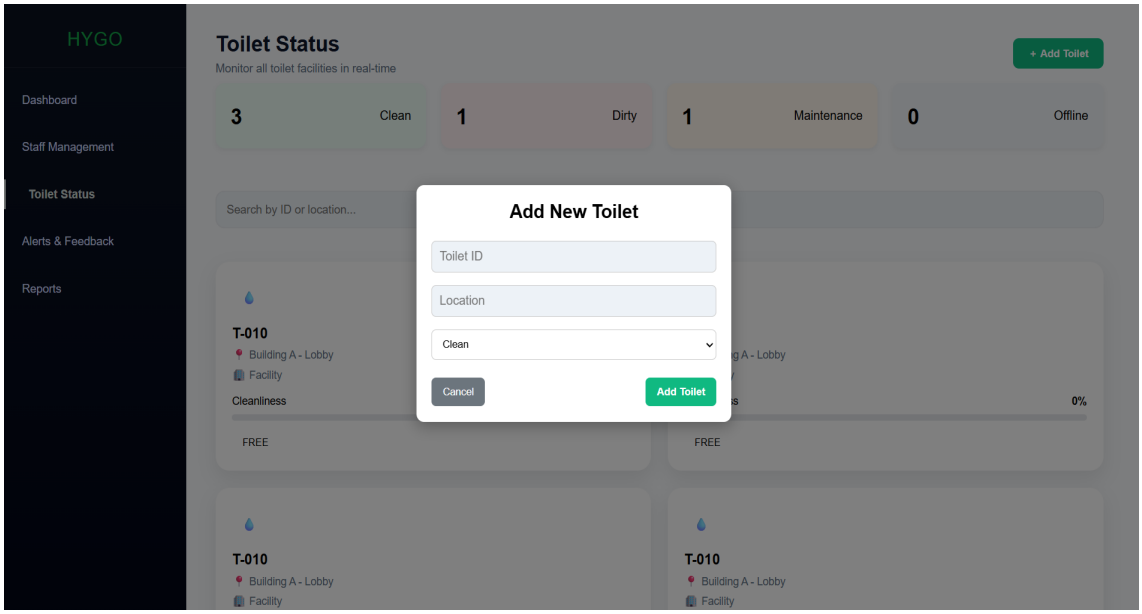

Figure 6.9: Add Toilets

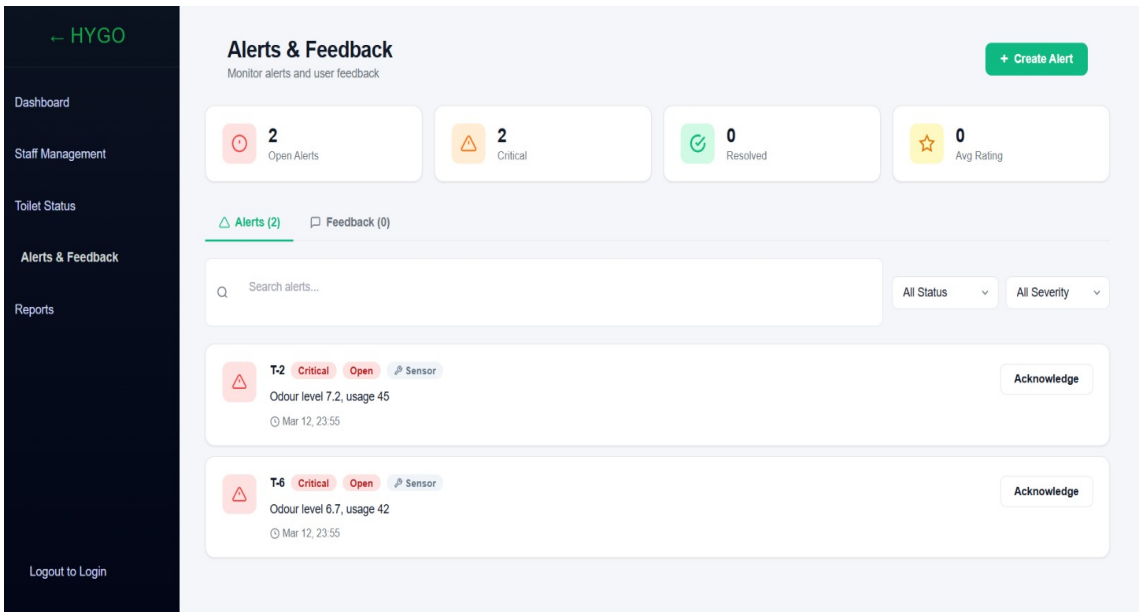

Figure 6.10: Alerts and Feedback

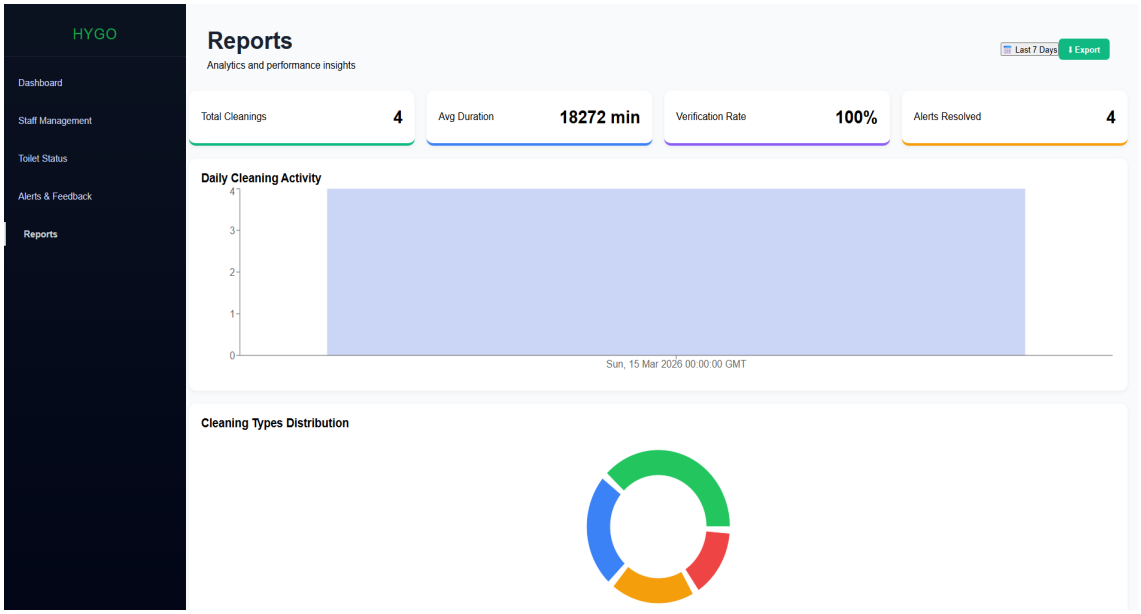

Figure 6.11: Reports

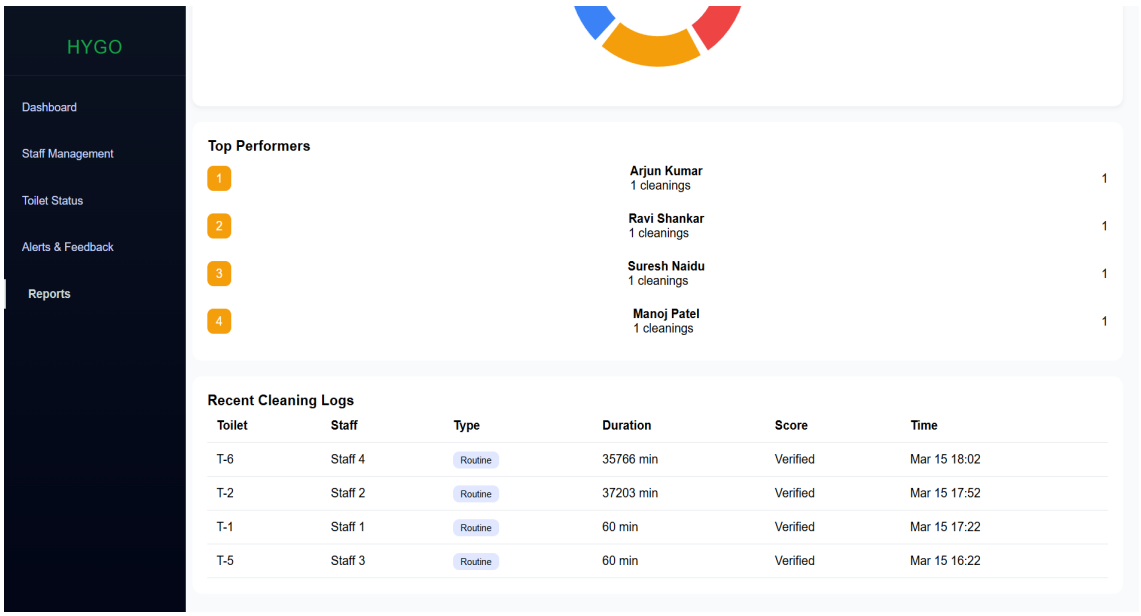

Figure 6.12: Reports

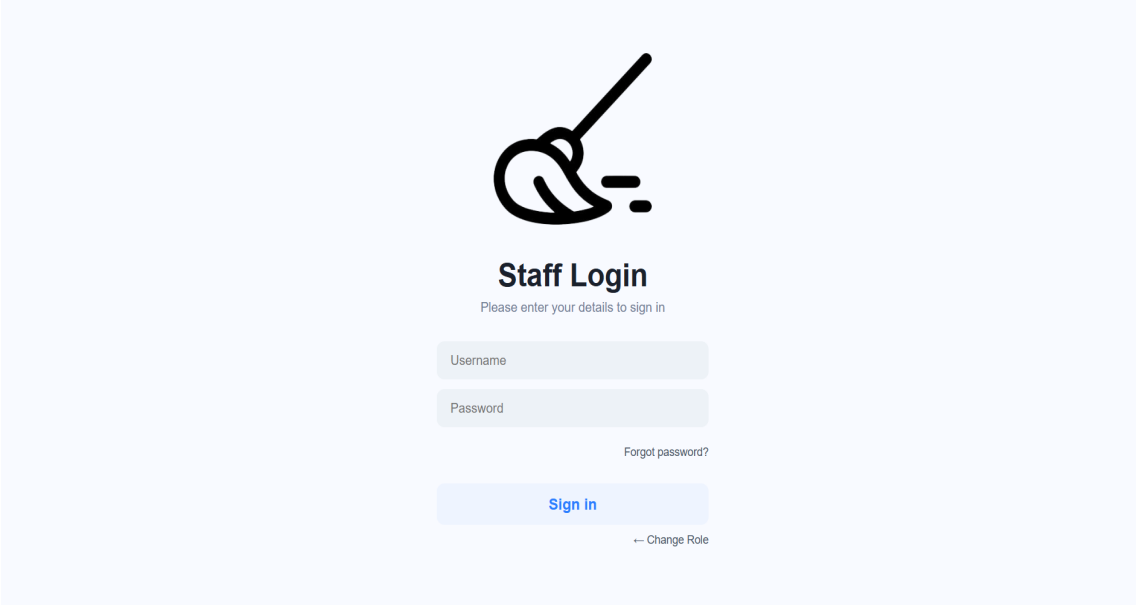

Figure 6.13: Staff Login

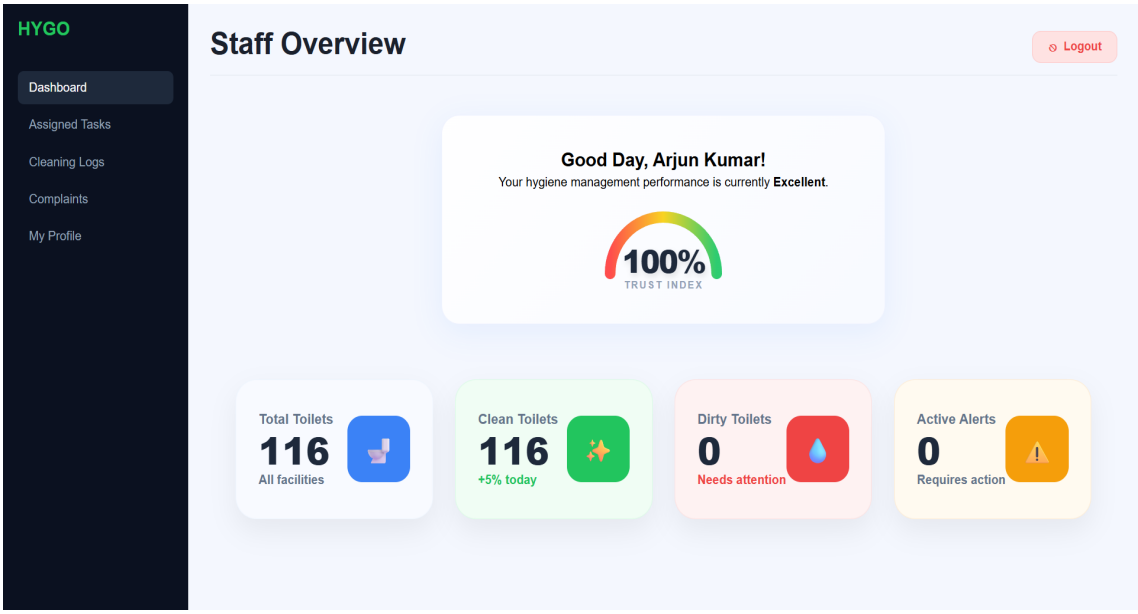

Figure 6.14: Staff Dashboard

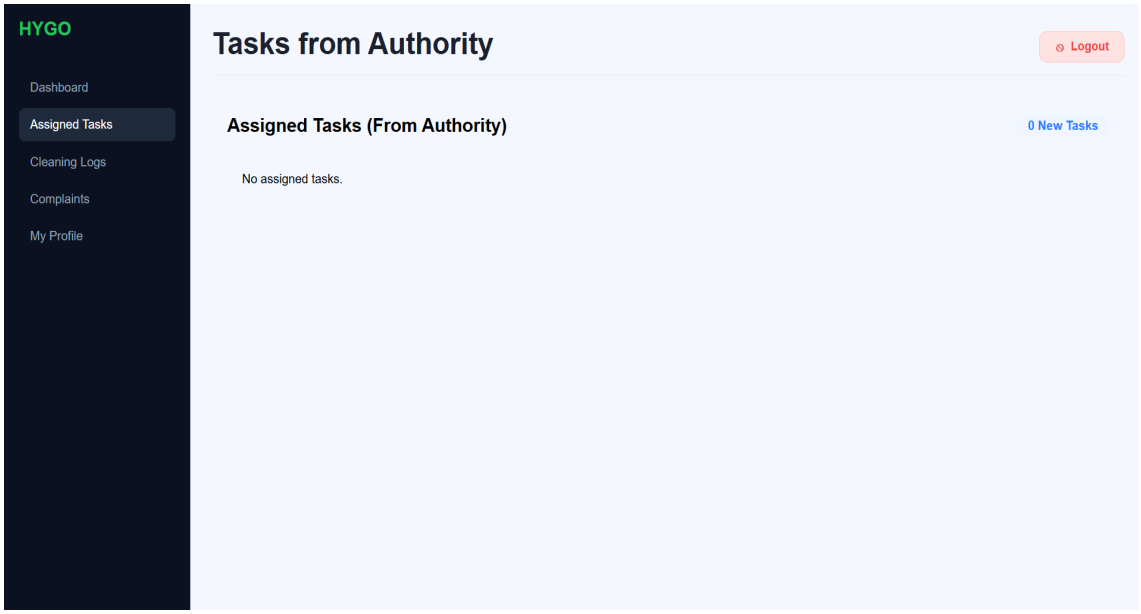

Figure 6.15: Assigning Staff

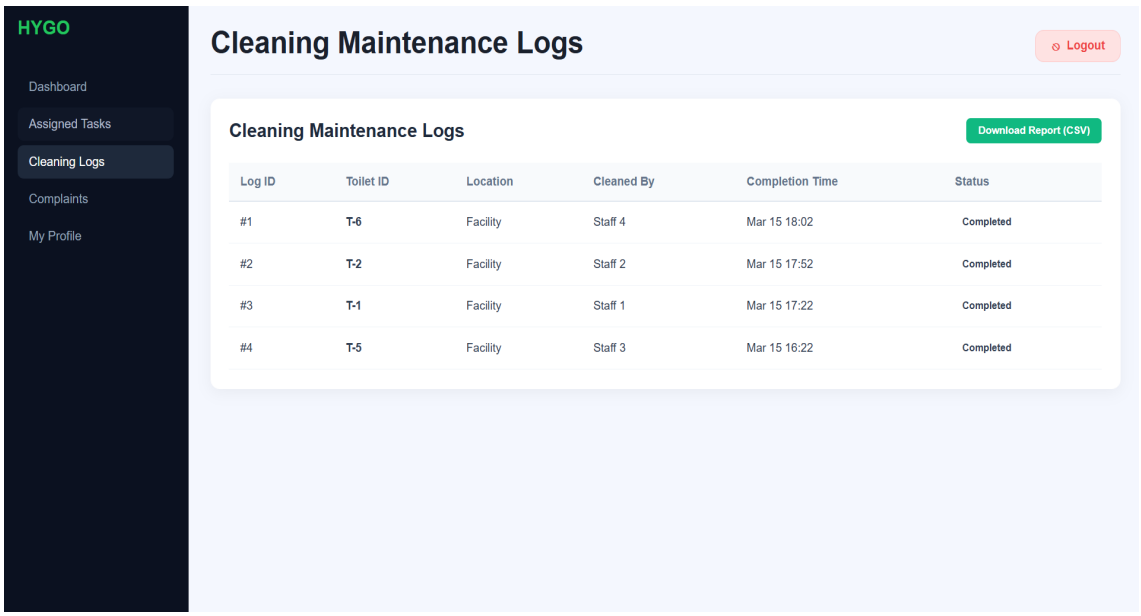

Figure 6.16: Cleaning Logs

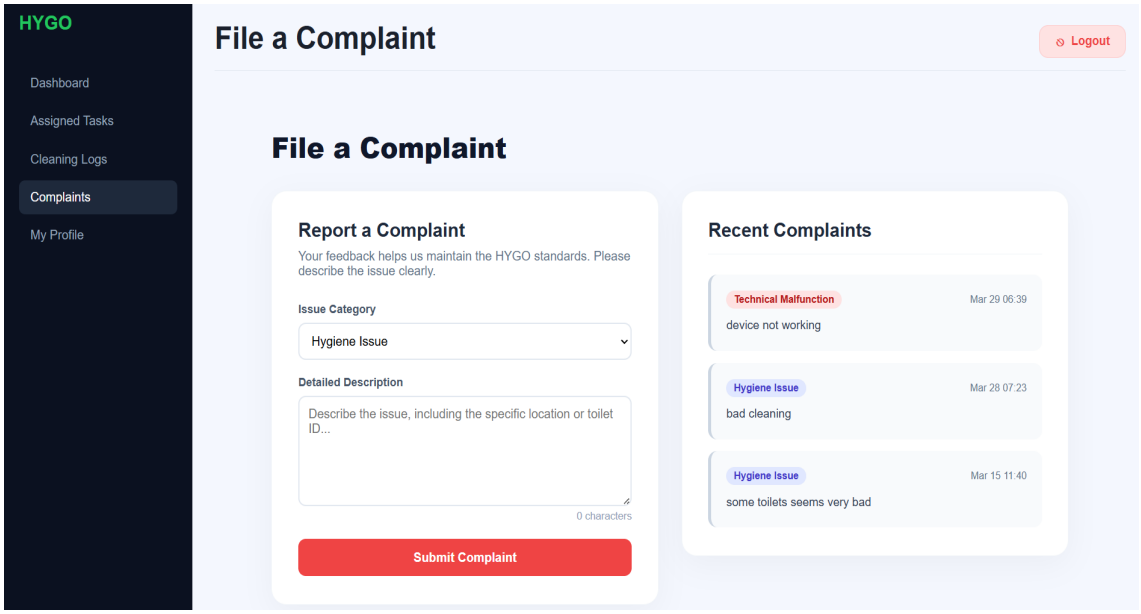

Figure 6.17: Staff Complaints

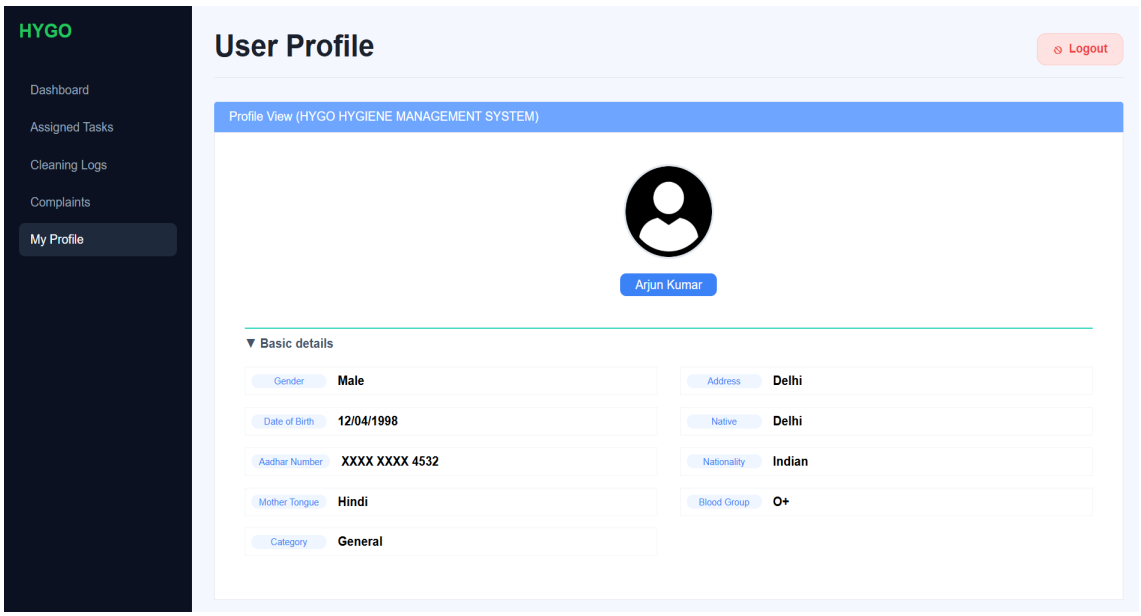

Figure 6.18: Staff Profile

Chapter 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

In conclusion, the HYGO – Smart Hygiene Management System provides an effective and intelligent solution for maintaining hygiene in public toilets through the integration of IoT technology, real-time monitoring and data-driven decision-making, successfully addressing the limitations of traditional fixed cleaning schedules by introducing a smart, demand-based approach that ensures cleanliness, efficiency and accountability. By incorporating IoT sensors such as the MQ-135 for odour detection and ultrasonic sensors for usage tracking, HYGO continuously monitors hygiene conditions and delivers accurate real-time data, while its automatic cleaning alert mechanism ensures timely action whenever hygiene levels fall below acceptable standards, improving overall sanitation and user experience. The system further enhances transparency and accountability through a staff monitoring and trust score mechanism that evaluates cleaning performance based on response time and efficiency, encouraging higher standards of work. Additionally, HYGO leverages predictive analytics and machine learning techniques such as Random Forest to analyze historical data and predict usage patterns, cleaning frequency and optimal cleaning times, enabling efficient resource planning and reduced unnecessary cleaning efforts. The web-based dashboard offers a user-friendly interface for administrators to monitor live data, manage alerts, analyze reports and oversee staff performance, while the inclusion of a complaints module ensures both automated and user-reported issues are addressed effectively. Overall, HYGO is a comprehensive smart hygiene management solution that combines automation, analytics and user interaction to enhance operational efficiency, promote better hygiene practices and contribute to improved public health, transforming traditional hygiene systems into a smarter, more sustainable and technology-driven approach.

7.2 Future Scope

The future of the HYGO – Smart Hygiene Management System offers significant opportunities for enhancement and expansion through advanced technologies. By integrating more sophisticated machine learning and artificial intelligence models, the system can improve the accuracy of hygiene prediction and enable more intelligent decision-making for cleaning operations. The incorporation of advanced IoT technologies can further enhance real-time monitoring by enabling the use of additional sensors such as humidity, temperature and ammonia detection, providing a more comprehensive assessment of hygiene conditions. Edge computing can also be introduced to process data locally on devices like ESP32, reducing latency and improving system responsiveness. In the future, HYGO can be extended into a mobile application to provide real-time notifications, alerts and monitoring capabilities directly to administrators and cleaning staff, improving accessibility and ease of use. Integration with smart city infrastructure can allow centralized monitoring of multiple public facilities, enabling large-scale deployment and management. The predictive analytics module can be further enhanced using deep learning techniques to provide more accurate forecasts of usage patterns and cleaning requirements. Personalized dashboards and automated scheduling systems can also be introduced to optimize resource allocation and reduce operational costs. Additionally, the system can incorporate advanced features such as image-based cleanliness detection using computer vision, automated complaint resolution systems and voice-based alerts for better user interaction. Integration with government and municipal systems can help in maintaining public hygiene standards more effectively. As technology continues to evolve, HYGO can transform into a fully autonomous and intelligent hygiene management platform, contributing to smarter cities, improved public health and sustainable sanitation practices.

References

[1] IEEE Xplore, "IoT-Based Smart Hygiene Monitoring System," [Online]. Available: https://ieeexplore.ieee.org/document/10544900

[2] A. Kumar, R. Singh, and P. Verma, "IoT-Based Smart Hygiene Monitoring System for Public Toilets," *IJERT*, 2022. [Online]. Available: https://www.ijert.org

[3] "Robotic Restroom Hygiene Solutions with IoT and Recurrent Neural Networks," 2024.

[4] Advaith et al., "IoT-Based Healthy Toilet System for Automated Cleanliness Tracking," 2023. [Online]. Available: https://www.scribd.com/document/644464032/Iot-Based-Healthy-Toilet

[5] "Smart Public Toilet Management and Monitoring System using IoT," [Online]. Available: https://www.researchgate.net/publication/378017906

[6] AskSensors Blog, "Air Quality Sensor MQ135 Cloud MQTT," [Online]. Available: https://blog.asksensors.com/air-quality-sensor-mq135-cloud-mqtt/

[7] MySQL Documentation, "MySQL Database Server," [Online]. Available: https://www.mysql.com

[8] Pallets Projects, "Flask Documentation," [Online]. Available: https://flask.palletsprojects.com/en/stable/

[9] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Computer Society.

[10] Mozilla, "REST API Design Best Practices," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Glossary/REST

[11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org

[12] "Joblib Documentation — Lightweight Pipelining in Python," 2024. [Online]. Available: https://joblib.readthedocs.io

[13] Meta Open Source, "React Documentation," 2024. [Online]. Available: https://react.dev

[14] Espressif Systems, "ESP32 Technical Reference Manual," 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

Appendix A

Implementation code fore core portion

A.1 Implementation of Hardware

```
#include <WiFi.h>
#include <HTTPClient.h>

// ---------------- WIFI ----------------
const char* ssid = "AndroidAP_6623";
const char* password = "aju756010";

// ---------------- BACKEND ----------------
const char* serverName = "https://hygo-smart-hygiene-management-system.onrender.com/api/

// ---------------- PINS ----------------
#define GAS_PIN 34
#define TRIG 5
#define ECHO 18

// ---------------- THRESHOLDS ----------------
#define DIST_OCCUPIED 25
#define GAS_HIGH_THRESHOLD 250
#define GAS_LOW_THRESHOLD 120
#define USAGE_LIMIT 30
#define MIN_USAGE_TIME 15000

// ---------------- VARIABLES ----------------
bool isOccupied = false;
```

```
bool alertTriggered = false;
bool wasGasHigh = false;   //  cleaning detection

int usageCount = 0;
long lastDistance = 100;
unsigned long enterTime = 0;

int toiletID = 10;   //  dynamic toilet support

// ---------------- GAS LEVEL ----------------
String getGasLevel(int gasValue) {
  if (gasValue < 180) return "LOW";
  else if (gasValue < 250) return "MEDIUM";
  else if (gasValue < 450) return "HIGH";
  else return "CRITICAL";
}

// ---------------- STATUS LOGIC ----------------
String getToiletStatus(int gasValue, int count) {

  if (gasValue >= GAS_HIGH_THRESHOLD || count >= USAGE_LIMIT)
    return "DIRTY";

  else if (gasValue >= GAS_LOW_THRESHOLD || count >= (USAGE_LIMIT / 2))
    return "MODERATE";

  else
    return "CLEAN";
}

// ---------------- DISTANCE ----------------
long getDistance() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG, HIGH);
```

```
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long duration = pulseIn(ECHO, HIGH, 30000);
  long distance = duration * 0.034 / 2;

  if (distance < 2 || distance > 400) return lastDistance;
  return distance;
}

// ---------------- SEND DATA ----------------
void sendData(int gasValue, String gasLevel, long distance, String status) {

  if (WiFi.status() == WL_CONNECTED) {

    HTTPClient http;
    http.begin(serverName);
    http.addHeader("Content-Type", "application/json");

    String json = "{";
    json += "\"toilet_id\":" + String(toiletID) + ",";
    json += "\"gas_value\":" + String(gasValue) + ",";
    json += "\"gas_status\":\"" + gasLevel + "\",";
    json += "\"distance\":" + String(distance) + ",";
    json += "\"status\":\"" + status + "\",";
    json += "\"count\":" + String(usageCount) + ",";
    json += "\"alert\":\"" + String(alertTriggered ? "ON" : "OFF") + "\"";
    json += "}";

    int response = http.POST(json);
    Serial.println("HTTP: " + String(response));

    http.end();
  }
}
```

```
// ---------------- SETUP ----------------
void setup() {
  Serial.begin(115200);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nConnected ");
}

// ---------------- MAIN LOOP ----------------
void loop() {

  int gasValue = analogRead(GAS_PIN);
  String gasLevel = getGasLevel(gasValue);
  long distance = getDistance();

  // -------- ENTRY --------
  if (!isOccupied && distance < DIST_OCCUPIED) {
    isOccupied = true;
    enterTime = millis();
  }

  // -------- EXIT --------
  if (isOccupied &&
     (millis() - enterTime >= MIN_USAGE_TIME || distance > DIST_OCCUPIED + 15)) {
    isOccupied = false;
    usageCount++;
  }
```

```
  lastDistance = distance;

  // -------- CLEANING DETECTION --------
  if (gasValue >= GAS_HIGH_THRESHOLD) {
    wasGasHigh = true;
  }

  if (wasGasHigh && gasValue < GAS_LOW_THRESHOLD) {
    usageCount = 0;        //  reset after cleaning
    wasGasHigh = false;
  }

  // -------- ALERT --------
  alertTriggered = (gasValue >= GAS_HIGH_THRESHOLD || usageCount >= USAGE_LIMIT);

  // -------- STATUS --------
  String status = getToiletStatus(gasValue, usageCount);

  // -------- SEND --------
  sendData(gasValue, gasLevel, distance, status);

  // -------- DEBUG --------
  Serial.print("GAS=");
  Serial.print(gasValue);
  Serial.print(" | STATUS=");
  Serial.print(status);
  Serial.print(" | COUNT=");
  Serial.print(usageCount);
  Serial.print(" | ALERT=");
  Serial.println(alertTriggered ? "ON" : "OFF");

  delay(2000);
}
```

A.2 Implementation of AI Model

```
A.2 AI Model { Advanced Core Logic
A.2.1 Data Preprocessing
#-------- Timestamp Feature Extraction -------
data['timestamp'] = pd.to_datetime(data['timestamp'])

data['hour'] = data['timestamp'].dt.hour        # Time of usage
data['weekday'] = data['timestamp'].dt.weekday  # Day pattern

#-------- Remove Unnecessary Columns -------
data = data.drop(columns=['timestamp','id'])
A.2.2 Feature Engineering
#-------- Priority Score Calculation -------
# Combines usage intensity and odour level
data['priority_score'] = (
    data['usage_count'] * 0.6 +
    data['odour_level'] * 40
)

#-------- Target Variable -------
y = data['priority_score']
A.2.3 Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
A.2.4 Model Training
model = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    random_state=42
)

model.fit(X_train, y_train)
A.2.5 Prediction
preds = model.predict(X_test)
```

```
A.2.6 Evaluation
mae = mean_absolute_error(y_test, preds)
r2 = r2_score(y_test, preds)
A.2.7 Model Saving & Loading
#-------- Save Model -------
joblib.dump(model, "hygo_model.pkl")

#-------- Load Model -------
def load_model():
    return joblib.load("hygo_model.pkl")
```

A.3 Implementation of Flask

```
from flask import Flask, request, jsonify
import mysql.connector
from datetime import datetime
import joblib
import numpy as np

app = Flask(__name__)

# Load ML model
model = joblib.load("hygo_model.pkl")

# DB connection
def get_db():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="YOUR_PASSWORD",
        database="hygo_db"
    )

# --------------------
# Login API
# --------------------
```

```
@app.route("/api/login", methods=["POST"])
def login():
    data = request.json
    username = data.get("username")
    password = data.get("password")

    db = get_db()
    cursor = db.cursor(dictionary=True)

    cursor.execute(
        "SELECT role FROM users WHERE username=%s AND password=%s",
        (username, password)
    )
    user = cursor.fetchone()

    db.close()

    if user:
        return jsonify({"success": True, "role": user["role"]})
    else:
        return jsonify({"success": False}), 401

# --------------------
# Cleaning Prediction
# --------------------
@app.route("/api/predict", methods=["POST"])
def predict():
    data = request.json

    usage = data.get("usage_count", 0)
    odour = data.get("odour_level", 0)
    hours_since_clean = data.get("hours_since_clean", 1)

    features = np.array([[usage, datetime.now().hour, hours_since_clean]])

    prediction = model.predict(features)[0]
```

```
    return jsonify({
        "predicted_cleaning_time": int(prediction)
    })

# --------------------
# Sensor Data API
# --------------------
@app.route("/api/data", methods=["POST"])
def sensor_data():
    data = request.json

    db = get_db()
    cursor = db.cursor()

    cursor.execute("""
        INSERT INTO sensor_data (toilet_id, odour_level, usage_count, timestamp)
        VALUES (%s, %s, %s, NOW())
    """, (
        data.get("toilet_id"),
        data.get("odour_level"),
        data.get("usage_count")
    ))

    db.commit()
    db.close()

    return jsonify({"message": "Data stored"})

# --------------------
# Run Server
# --------------------
if __name__ == "__main__":
    app.run(debug=True)
```